

KAMP 제조AI데이터셋 활용 권한

No. GB20250818-PU1755498594-243790



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 표면처리 자원 최적화 AI 데이터셋, KAIST(울산과학기술원, (주)에이비에이치), 2022.12.23., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Surface treatment resource optimization AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 표면처리 자원 최적화 AI 데이터셋

업종 : 뿌리(표면처리) | **목적** : 자원 최적화 | **유형** : csv

알고리즘 : Randomforest

사용조건 : 콘텐츠 변경 허용

제공기관 : KAIST (수행기관 : 울산과학기술원/(주)에이비에이치)

등록일 : 2022-12-23

이용자 정보

회원 ID : 01soju

다운로드일 : 2025-08-18 17:25:30

활용목적 : 공부

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

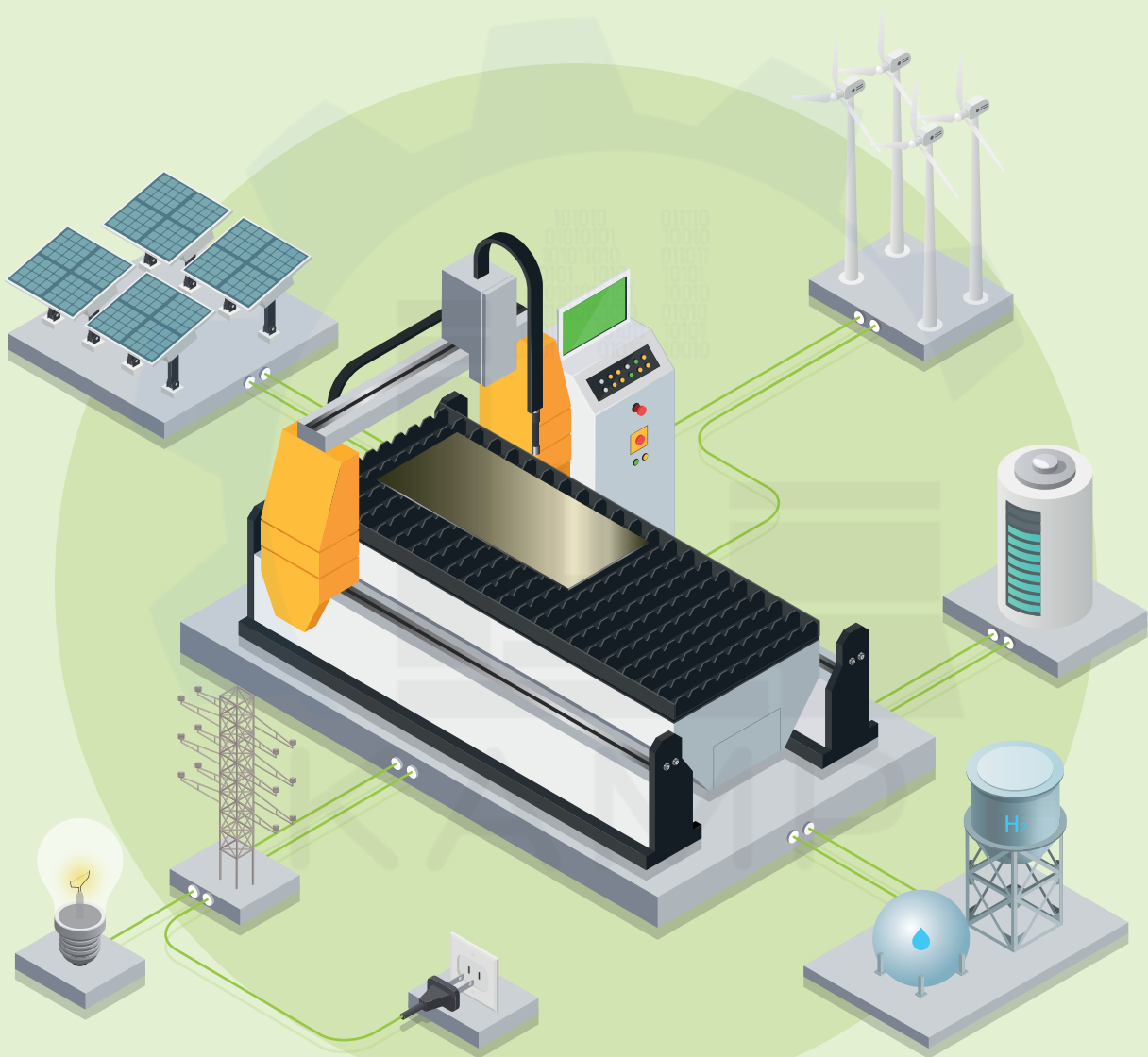
운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「표면처리 자원최적화 AI 데이터셋」

분석실습 가이드북



「표면처리 자원최적화 AI 데이터셋」

분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	07
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	08
2.1 제조데이터 소개	08
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	12
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	14
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 라이브러리/데이터 불러오기	
[단계 ②] 데이터 종류 및 개수 확인	
[단계 ③] 데이터 정제 (전처리)	
[단계 ④] 훈련/테스트 데이터 분리	
[단계 ⑤] 랜덤 포레스트 모델 구축 및 학습	
[단계 ⑥] 선형 계획 모델 수립	
[단계 ⑦] 최종 결과 도출	
3. 유사 타현장의 분석 적용	32
부 록	
분석환경 구축을 위한 설치 가이드	33
데이터 품질 전처리 [실습 코드]	46



「표면처리 자원최적화 AI 데이터셋」 분석실습 가이드북

- 필요 SW : Python, Anaconda - Jupyter Notebook
- 필요 패키지 : Pandas, matplotlib, scikit-learn
- 분석 환경 : [운영체제] Ubuntu 14.0 이상, [CPU] Intel Xeon 2.3 GHz, [RAM] 13GB, [GPU] Tesla K80
- 필요 데이터 : Resource_Management_Process.csv

1 분석요약

No	구 분		내 용
1	분석 목적 (현장 이슈, 목적)		현 공정의 경우 공정라인이 존재하지만 공정 라인에서 사용되는 에너지 및 공정에 필요한 자원의 사용량을 컨트롤하고 있지 못하고 있는 상황이며, 라인이 존재함에도 불구하고 연속공정 중간에 투입되는 인력이 존재하여 공정을 진행함에 있어 불필요한 자원이 있다고 판단된다. 이를 해결하기 위해 AI 분석을 통하여 사용되는 자원에 대한 분석을 진행하고, 불필요하게 사용되는 자원을 파악하여 공정에 사용되는 자원을 절감하고 원가 절감을 진행할 수 있도록 시스템 구축이 필요하다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : Datetime, Power_Cost, Worker_Power 외 6개 2) 데이터 수집 방법 : 생산업체의 생산량, 공정 변수, 투입인력에 대한 Data 3) 데이터셋 파일 확장자 : CSV
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 총 220,311개 (row수 24,479개*column수 9개) - 데이터셋 총량 : 1.33MB
4	분석적용 알고리즘	알고리즘	Randomforest
		알고리즘 간략소개	Randomforest는 트리와 트리 내에 계층구조로 이루어진 노드들과 에지들의 집합으로 이루어져 있다. 다양하게 만들어진 노드를 활용하여 분석 data 파라미터에 대한 조정이 용이하며, data의 scale 조정이 불필요하므로 다양한 scale이 섞여있는 데이터 분석에 용이하다. 이를 활용하여 크기와 종류가 다양하게 분포되어 있는 인적 및 물적 자원에 대한 분석을 통하여 불필요하게 사용되는 자원을 찾아내고 최적화를 진행한다.
5	분석결과 및 시사점		- 과거 스케줄 데이터를 인공지능 모델인 랜덤 포레스트 모델을 학습하여 작업량에 따라 필요한 노동력을 예측하는 모델을 훈련하였다. 이를 수치화시키는 선형계획모델을 통해 당일 작업량에 필요한 공정 인원을 도출한다. - 최적 공정 투입 인원을 도출함에 따라 공정운영 비용감소 효과를 기대할 수 있다.

2 분석 실습

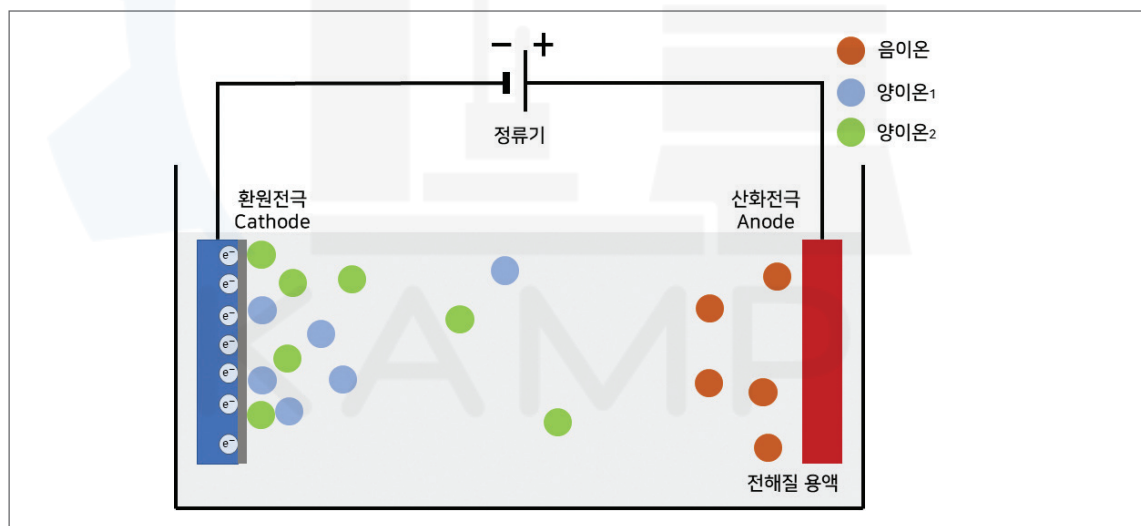
1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

• 전기아연도금 공정

- 전기아연도금 공정은 표면처리 공정의 일종으로, 전류가 통하는 금속 모재를 도금액에 침적시켜, 전류를 통과시켜 모재 표면에 얇은 아연 피막층을 형성시키는 공정이다.
- 도금 공정은 연속공정으로 자동화 설비에 의하여 표면 세척부터 도금 후처리에 이르는 과정이 진행된다.
- 이후 품질검사를 거쳐 열풍건조 과정에 따라 공정이 완료된다.
- 부품 및 소재류의 물리 화학적 특성을 강화하는 특징에 따라 공정의 대상이 되는 제품 종류가 100가지 이상에 이르는 특징을 가진다.
- 이에 따라 제품 종류별 공정 조건의 변동이 발생하며, 이 과정에서 작업자의 노하우에 의존하는 공정변수 선정이 진행되어 비효율이 발생하기 쉽다.

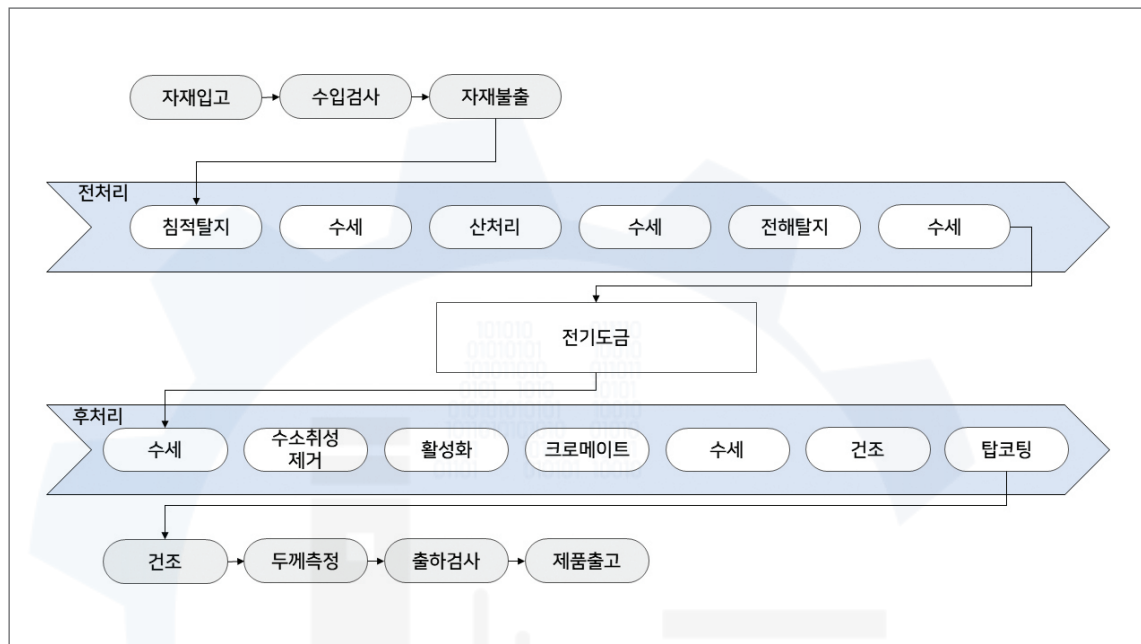


[그림 1] 도금공정 개요

• 전기아연도금 설비

- 해당 기업의 공정은 전기아연도금을 기반으로 한 기업 특유의 기술을 적용한 자동화 설비를 활용하며, 해당 과정에서 센서 및 PLC로부터 데이터가 자동적으로 수집된다.
- 첫 단계로 도금체 표면에 묻어 있는 이물질을 제거하기 위하여 산성 물질에 담구어 수세 작업을 진행한다.

- 1차 수세 작업이 마무리 되면 기포를 활용하여 미세한 이물질과 기름때를 지우는 전해탈지를 진행하며, 이것이 마무리된 후 전기아연 도금을 진행한다.
- 전기아연 도금이 마무리 되고 난 후에 아연도금 표면의 부식을 늦추기위한 크로메이트 코팅을 진행한다.
- 이후 마지막으로 코팅된 부분을 건조하여 최종적인 화학 결합을 하여 표면을 안정시키는 작업을 진행한다.



[그림 2] 자동화 도금공정 프로세스

2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 공정 대상인 피도금체가 하나로 통일되지 못하는 상황으로 인하여 전류값과 시간 등 주요 공정변수에 대한 가이드라인이 제공되지 못하고 있다.
- 이는 도금업 특성상 수주 제품의 종류가 다양해져 생기는 불가피한 상황으로, 제품의 형태와 표면적, 질량 등 다양한 요인에 따라 달라지는 공정 설정을 결정하기 어려워진다.
- 이로 인하여 전력 소모와 원재료 소모, 불량 발생에 따른 모재의 폐기 등 자원의 낭비가 발생하고 있으며 특히 고부가가치성 소재의 도금 과정에서는 치명적으로 작용하고 있다.

• 문제해결 장애요인

- 데이터의 축적은 지속적으로 이루어지고 있으나 새로운 피도금체의 추가와 공정 외 변수인 환경변수의 작용 등 다양한 변인에 의하여 최적 공정변수를 알아내기 어렵다.

- 특히나 Manpower 같은 인력에 대한 데이터는 수식으로 이루어져 있다고 하더라도 결국에는 일반적으로 수집할 수 있는 데이터와 달리 주관적인 입장이 들어갈 수 밖에 없기 때문에 데이터에 대한 신뢰도가 떨어지며 이를 해결하기 위한 방안이 필요하다

• 극복 방안

- AI 분석을 통하여 사용되는 자원에 대한 분석을 진행하고, 불필요하게 사용되는 자원을 파악함으로써 공정에 사용되는 자원을 절약하고 원가 절감을 진행한다.
- 이 과정에서 장애 요인인 다양한 변인에 대하여 빠른 학습 및 분석을 제공할 수 있는 Random Forest를 활용, 제품군별 최적 자원의 산출을 진행한다.

1.2 분석 목표

1) 분석 목표

- 각각의 제품별로 공정조건 및 환경조건, 품질조건에 대한 AI 분석을 통하여 사용되는 자원에 대한 분석을 진행하고, 최적화된 자원값을 산출하고자 한다.

2) 데이터 정의 및 소개

- 해당 분석에서 사용되는 공정 데이터들은 실제 21년 2월 8일부터 21년 10월 21일까지, 추가로 22년 01월 13일부터 22년 07월 14일까지의 공정 데이터(생산량, 온도, 습도, 노동력) 및 에너지 데이터(전력비용, 매몰비용, 에너지 효율)를 수집하였으며, 통합 데이터셋을 구축하여 활용한다.

3) 데이터 분석 기대효과

- 데이터 분석을 통해 생산과정에서의 에너지 소비량을 분석 · 모니터링하고, 이를 기반으로 최적화된 자원 소모값을 산출함으로써 원가 절감을 달성할 수 있을 것으로 기대된다.
- 최적 자원 소모값을 산출 · 제어함으로써 인력의 중간투입을 최소화하여 라인 자동화의 효율을 더욱 끌어올릴 수 있을 것으로 판단된다.

4) 시사점(implication) 요약기술

- 현재 설비 내 공정에서 스케줄에 따른 업무량을 바탕으로 제조 인력을 어림잡아 투입한다. 이러한 인원 투입방식은 제조 업무 스케줄에는 거의 지장이 없지만, 때때로 과도한 인력 투입으로 인해 투입된 인원 대비 생산량 저조 등의 현상이 발생할 수 있다. 업무



스케줄과 업무 타입에 따른 다양한 변수를 파악하고, 필요한 생산력을 예측하는 AI모델을 개발하고, 이를 바탕으로 필요한 인력을 정의하는 선형계획 모델을 정의한다.

- 본 분석은 설비 내 공정에서 생산량, 온도, 습도, 전력비용, 요일, 노동력 등의 데이터로부터 AI 모델 개발과 제조공정 상에서의 적용 및 검증을 통해 중소기업에서 데이터 및 AI를 적용하여 실질적인 인력 투입을 사전에 예측하여 관련된 비용을 사전에 예상 또는 비용 절감에 기여할 수 있다는 점에서 시사하는 바가 크다고 판단된다.

2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- 제조 분야 : 표면 처리
- 수집 장비 : 설비 내 공정 데이터, 품질 데이터, 원자재 및 중간재 데이터
- 수집 기간 : 2021년 02월 08일 ~ 2021년 10월 21일
- 수집 주기 : 0.5 sec
- 데이터 이름
 - 데이터 : Resource_Management_Process.csv

2) 데이터 유형/구조

- 데이터셋 구조 : 테이블 형식(Tabular)
 - 실제 공정에서 발생한 데이터를 받아서 csv 형태로 저장
- 데이터 개수
 - 공정 데이터 : Column 수 9개, Row 수 24,479개

A	B	C	D	E	F	G	H	I
2021-10-21 16:00	463	23.5	90.4	167.2	Thursday	2.31	1	201
2021-10-21 16:15	464	21.9	93.2	167.2	Thursday	2.33	1	207
2021-10-21 16:30	462	22.6	88.1	167.2	Thursday	2.33	1	208
2021-10-21 16:45	461	22	90.1	167.2	Thursday	2.33	1	202
2021-10-21 17:00	172	21.8	83.8	167.2	Thursday	0.59	1	185
2021-10-21 17:15	173	22.8	87	167.2	Thursday	0.61	1	157
2021-10-21 17:30	173	22.5	89	167.2	Thursday	0.62	1	139
2021-10-21 17:45	172	22.5	89.2	167.2	Thursday	0.62	1	178
2021-10-21 18:00	358	21.5	86.7	167.2	Thursday	1.66	1.5	204
2021-10-21 18:15	356	22.6	86.1	167.2	Thursday	1.68	1.5	200
2021-10-21 18:30	359	21.4	89.4	167.2	Thursday	1.66	1.5	204
2021-10-21 18:45	357	22.1	88.9	167.2	Thursday	1.67	1.5	188
2021-10-21 19:00	487	23.7	82.7	167.2	Thursday	2.57	1.5	182
2021-10-21 19:15	490	22.4	77.9	167.2	Thursday	2.58	1.5	194
2021-10-21 19:30	488	21.9	80.2	167.2	Thursday	2.59	1.5	191
2021-10-21 19:45	490	22.4	82.2	167.2	Thursday	2.56	1.5	212
2021-10-21 20:00	126	22.4	72.2	167.2	Thursday	0.22	1.5	177
2021-10-21 20:15	126	22.5	72.9	167.2	Thursday	0.24	1.5	172
2021-10-21 20:30	123	22.6	71.7	167.2	Thursday	0.23	1.5	173
2021-10-21 20:45	124	23.3	71.1	167.2	Thursday	0.21	1.5	171
2021-10-21 21:00	150	22.3	68.6	167.2	Thursday	0.42	1.5	167
2021-10-21 21:15	150	23.9	67.1	167.2	Thursday	0.42	1.5	177
2021-10-21 21:30	149	22.6	68.2	167.2	Thursday	0.44	1.5	176
2021-10-21 21:45	151	22.5	66.3	167.2	Thursday	0.42	1.5	170
2021-10-21 22:00	130	22.1	73.1	167.2	Thursday	0.3	1.5	153
2021-10-21 22:15	131	23.1	75.9	167.2	Thursday	0.29	1.5	145
2021-10-21 22:30	131	22.1	71.1	167.2	Thursday	0.28	1.5	149
2021-10-21 22:45	129	23.6	74	167.2	Thursday	0.3	1.5	162
2021-10-21 23:00	217	22.5	74.6	167.2	Thursday	1.07	1.5	134
2021-10-21 23:15	217	22.2	71.3	167.2	Thursday	1.05	1.5	157
2021-10-21 23:30	213	23.1	72.7	167.2	Thursday	1.05	1.5	163

[그림 3] 공정 Data 예시

• 제조AI데이터셋 주요 변수 정의

- Resource_Management_Process.csv

속성(column)	설명	데이터형	개수
DateTime	측정 시 시간을 분 단위까지 기록(H:MM)	str	24,479
Production	생산량	int	24,479
Temperature	온도	float	24,479
Humidity	습도	float	24,479
Power_Cost	전력 비용	float	24,479
DoW	요일	str	24,479
Worker_Power	노동력	float	24,479
Man_Cost	매물 비용	float	24,479
Power_Usage	설비들의 에너지 효율 정도	int	24,479



• 주요 변수 기술 통계

	속성	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
Resource_ Management_ Process	DateTime	str	24,479	-	-	-	-	-
	Production	int	24,479	231.56	214.89	112	126	2573
	Temperature	float	24,479	16.33	9.68	-13.7	17.8	36
	Humidity	float	24,479	63.23	24.22	-2	67.4	97
	Power_Cost	float	24,479	162.53	30.83	109.8	167.2	191.6
	DoW	str	24,479	-	-	-	-	-
	Worker_Power	float	24,479	1.04	1.99	0.1	0.25	48.5
	Man_Cost	float	24,479	1.31	0.24	1	1.5	1.5
	Power_Usage	int	24,479	136.47	58.11	39	147	270

속성	명칭
독립변수	Production, Temperature, Humidity, Power_Cost, Worker_Power
종속변수	Man_Cost, Power_Usage

3) 데이터 품질 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터들은 값이 의미 없거나 누락 및 오타가 발생하여 데이터 품질이 떨어진다.
- 아무리 좋은 분석 방법론을 가지고 있더라도 품질이 낮은 데이터를 이용하면 좋은 결과를 얻을 수 없으므로, 데이터 품질 전처리는 데이터 분석에서 가장 중요한 단계이다.
- 따라서, 데이터들의 6가지 품질지수를 파악하고, 데이터 전처리를 통해 품질지수를 향상한다.

• 데이터 품질 지수

- 완전성(Completeness) : 필수항목에 누락이 없어야 한다.
- 유일성(Uniqueness) : 데이터 항목은 유일해야 하며 중복돼서는 안 된다.
- 유효성(Validity) : 데이터 항목은 정해진 데이터 유효범위 및 도메인을 충족해야 한다.
- 일관성(Consistency) : 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
- 정확성(Accuracy) : 실제 존재하는 객체의 표현 값이 정확히 반영되어야 한다.



- 무결성(Integrity) : 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 유일성, 일관성, 유효성이 보호돼야 한다.

• 데이터 품질 지수 세부 설명

$$\text{완전성 품질지수} = (1 - (\text{결측치 데이터 수} / \text{전체 데이터 수})) \times 100$$

- pandas 라이브러리의 DataFrame 객체 isnull() 함수를 이용해 결측치 데이터 수를 구하고, len() 함수를 이용해 전체 데이터 수를 구한 후 완전성 품질지수를 측정한다.

$$\text{유일성 품질지수} = (\text{유일한 데이터 수} / \text{전체 데이터 수}) \times 100$$

- pandas 라이브러리의 DataFrame 객체 unique() 함수를 이용해 유일한 데이터 수를 구하고, 유일성 품질지수를 측정한다.

$$\text{유효성 품질지수} = (\text{유효성 만족 데이터 수} / \text{전체 데이터 수}) \times 100$$

- 데이터 측정 시각에 대해 날짜 시간의 표현이 “연-월-일 시:분”으로 정해진 datetime 형식인지 검사하고 유효성 품질지수를 측정한다.

$$\text{일관성 품질지수} = (\text{일관성 만족 데이터 수} / \text{전체 데이터 수}) \times 100$$

- 데이터의 각 column 별 데이터형이 일관성 있는지 검사하고 일관성 품질지수를 측정한다.

$$\text{정확성 품질지수} = (1 - (\text{정확성 위배 데이터 수} / \text{전체 데이터 수})) \times 100$$

- AI 모델 학습에 사용되는 자원 관리 데이터에 대해 측정 기간이 정확한지 검사하고 정확성 품질지수를 측정한다.

$$\text{무결성 품질지수} = (\text{유일성, 유효성, 일관성 지수중 100\%가 되는 지수 개수} / 3) \times 100$$

- 계산된 유일성, 유효성, 일관성 품질지수를 이용해 무결성 품질지수를 측정한다.

$$\text{가중치 지수} = \text{품질지수} \times \text{가중치}$$

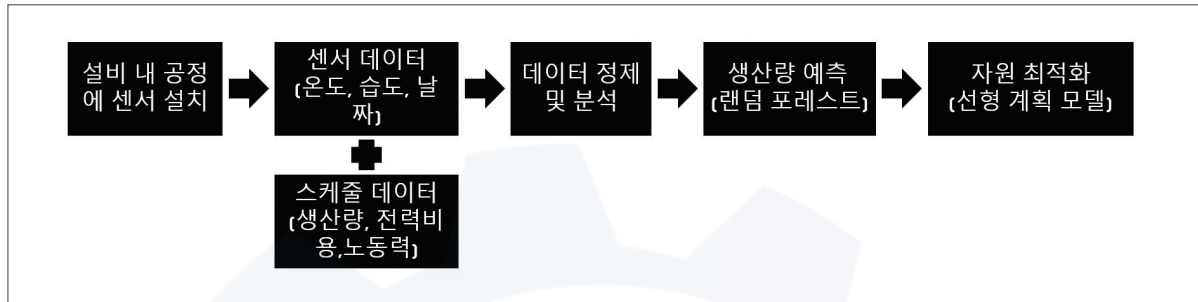
- 완전성, 유일성, 유효성, 일관성, 정확성, 무결성 품질지수에 각 0.2, 0.2, 0.2, 0.15, 0.15, 0.1의 가중치를 부여하고 가중치 지수를 계산한다.

데이터 품질지수 = Σ 가중치 지수

- 각 품질지수와 가중치를 이용해 종합적인 데이터 품질지수를 측정한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



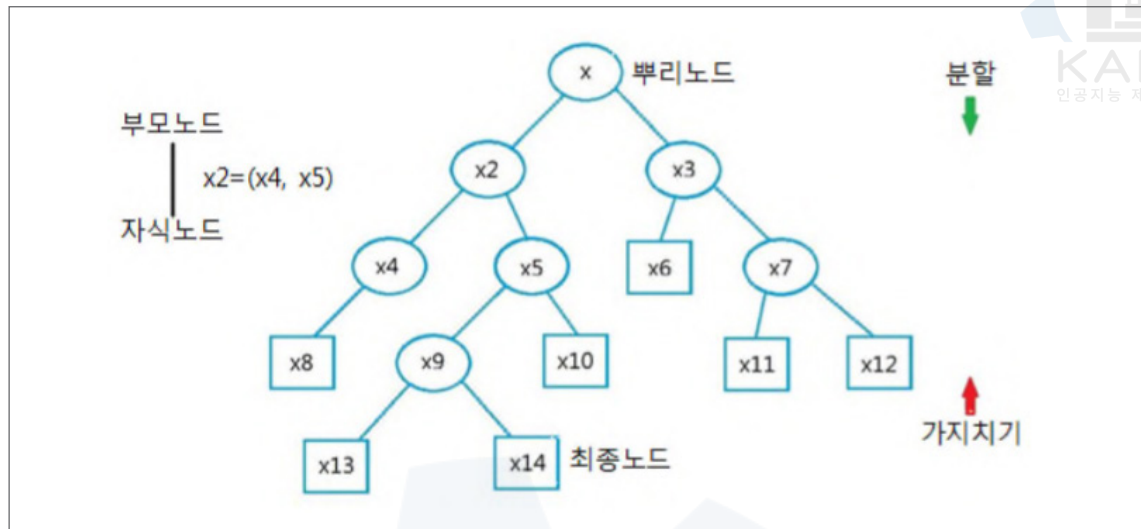
2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유

- 랜덤 포레스트란 여러 개의 결정트리(Decision Tree)를 이용하여 학습시키는 방법이다.
- 직관적인 머신러닝 알고리즘으로 분류와 회귀 모델 모두 사용할 수 있지만, 최대수요 전력을 예측하기 위해 여기에서는 회귀 모델 사용을 한다.
- 최대수요전력과 관련된 데이터 샘플링 시 일부 변수들만 무작위하게 선별하여 각 모델이 다른 변수를 학습할 수 있도록 한다. 회귀 모델을 사용하였을 때 모델을 어떻게 정의하고 접근하는지 아래에서 서술한다.

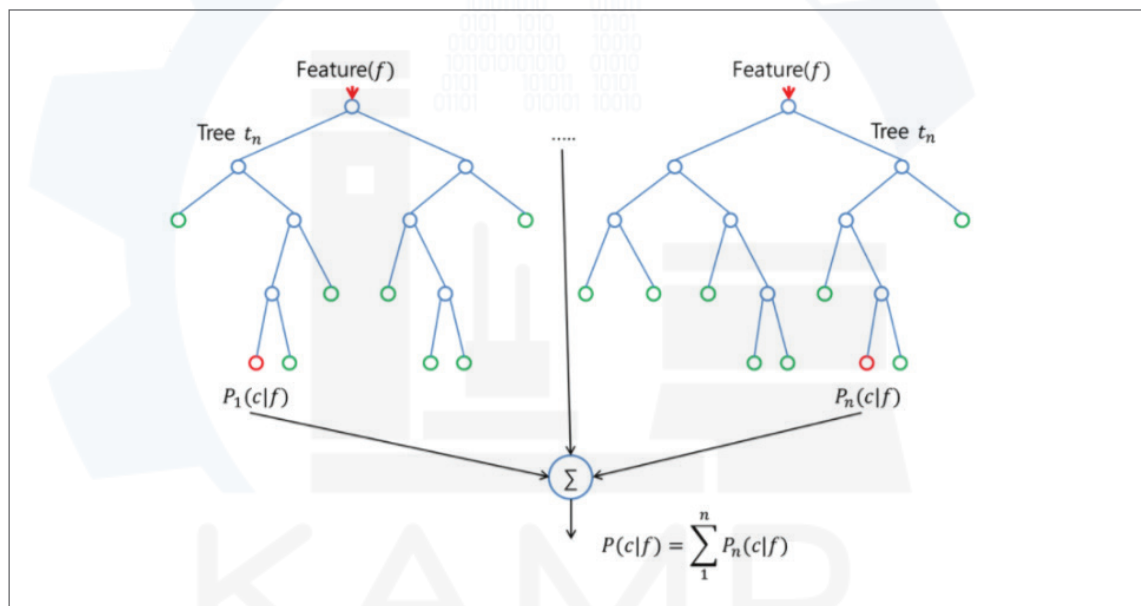
• 적용하고자하는 AI 분석 방법론(알고리즘)의 구체적 소개

- 랜덤 포레스트 (Random Forest)란 앙상블 (Ensemble) 학습 방법의 일종으로 분류와 회귀 분석에 모두 사용되는 직관적인 머신러닝 알고리즘이다.
- 여러 개의 결정 트리(Decision tree)를 생성하여 새로운 데이터를 각 트리에 통과시켜, 각 트리가 분류한 결과에서 투표를 실시하여 가장 많이 득표한 결과를 최종 분류 결과로 선택한다.
- 랜덤 포레스트의 기본이 되는 의사결정 나무 구조는 다음과 같다.



[그림 4] 의사결정 나무 구조 예시

- 여러 개의 결정 트리를 활용한 랜덤 포레스트의 구조는 다음과 같다.



[그림 5] Random Forest의 앙상블 machine learning 모델

• AI 분석 방법론(알고리즘) 구축 절차

- 제조공정 과정에서 제공되는 제조 스케줄 데이터를 이용하여, 품목별로 필요한 제조 공정의 전력을 사용량을 예측하고, 최적화 모델을 이용하여 전력 사용량에 따른 생산 필수량을 정의하고 최적화하여, 최적의 솔루션을 도출하는 것을 목표로한다. 이에 다음에 따라 분석을 수행한다.

- ① 패키지/데이터 불러오기 : 분석에 필요한 파이썬 패키지를 선언하고, 제공된 테이블 형태의 데이터셋을 읽어 들인 후, 확인한다.
- ② 데이터 탐색 : 데이터셋의 구성과 통계치를 확인한다.

- ③ 데이터 전처리 : 데이터셋 내에서 AI 모델에 학습시킬 레이블 데이터와 학습데이터를 분리하고, 최적화 모델의 수식에 사용될 데이터도 따로 분리한다. 그 후 데이터셋 내에서 AI 모델에 학습시킬 레이블 데이터에서 일부 숫자형 데이터가 아닌 데이터(날짜 등등)를 숫자로 변형한다. 그 후, 학습데이터를 분리하고, 최적화 모델의 수식에 사용될 데이터도 따로 분리한다.
- ④ AI 모델 구축 : Sklearn 라이브러리를 이용하여 AI 모델을 정의한다.
- ⑤ AI 모델 학습 : 전처리한 데이터를 이용하여 AI 모델을 학습한 후 결과를 확인한다.
- ⑥ AI 모델 평가 : 학습된 AI 모델을 이용하여, 테스트 스케줄 데이터에 대해 전력 사용량을 예측하고, 모델을 평가한다.
- ⑦ 최적화 모델 구축 및 적용 : 스케줄 데이터의 변수들을 이용하여 당일 제조 공정 작업에 필요한 생산력을 정의하고, 생산력을 만족시키기 위한 인원수를 예측하기 위해 AI 모델의 예측값과 인원값 변수를 이용한 최적화 모델을 구축하여 최적해를 찾는다.
- ⑧ 결과 분석 : 일련의 과정을 통해 도출된 결과를 분석하고, 활용방안 및 가능성을 설명한다.

2.3 분석 체형

1) 필요 SW, 패키지 설치 방법 및 절차 가이드

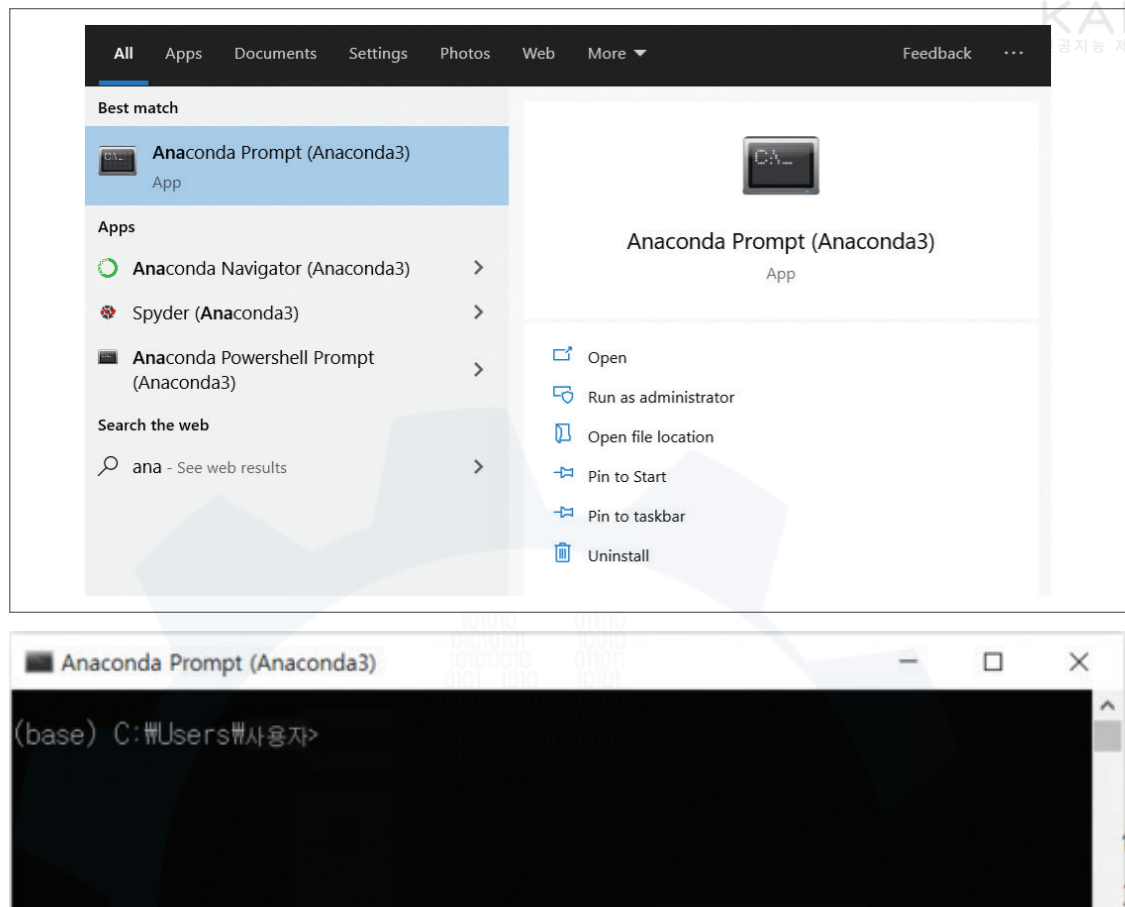
▶ 필요 SW 설치

- 프로그래밍 언어인 파이썬(Python)을 사용하며 개발 환경으로는 주피터 노트북(Jupyter Notebook)을 활용한다. 관련 SW 설치는 3. 부록 [분석 환경 구축을 위한 설치 가이드]를 참고하면 된다.

▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

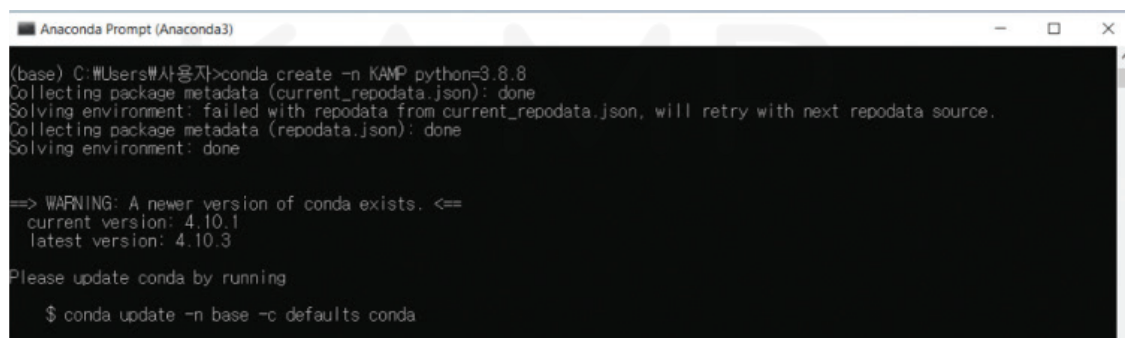
- 아나콘다 프롬프트 실행



- 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

```
conda create -n KAMP python=3.8.8
```



...

```
Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.
```

...



```
done
##
## To activate this environment, use
##
## $ conda activate KAMP
##
## To deactivate an active environment, use
##
## $ conda deactivate
##
(base) C:\Users\사용자>
```

- 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
conda activate KAMP
```

```
(base) C:\Users\사용자>conda activate KAMP
(KAMP) C:\Users\사용자>
```

- 분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\사용자>conda deactivate
(base) C:\Users\사용자>
```

- 주피터 노트북에서 가상 환경 실행

주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 비활성화(종료) 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

```
pip install ipykernel

# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
python -m ipykernel install --user --name KAMP --display-name "KAMP"

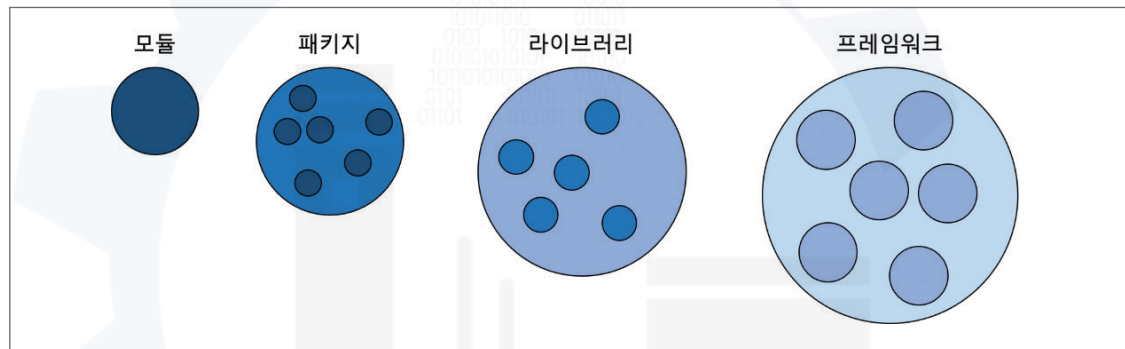
# jupyter notebook 실행
jupyter notebook
```



▶ 패키지 설치 가이드

- 패키지 설치 전 파이썬 관련 용어 정리

모듈 (module)	- 함수나 변수 또는 클래스를 모아 놓은 파일 - 파이썬 확장자(.py) 파일에 특정 함수, 변수, 클래스 등이 저장된 코드이다.
패키지 (package)	- 여러 모듈의 집합 - 하나의 디렉토리(경로)에 놓인 모듈들의 집합을 의미하며, 일반적으로 <code>__init__.py</code>라는 패키지 초기화 파일이 존재한다. - 예) NumPy : 수치해석 패키지, pandas : 데이터 분석 패키지
라이브러리 (library)	- 여러 패키지의 집합 - 파이썬을 설치할 때 기본적으로 설치되는 라이브러리를 표준라이브러리라고 한다. 파이썬 공식이 아닌 외부에서 개발한 모듈과 패키지를 묶어 외부 라이브러리라고 한다. - 예) matplotlib : 데이터 시각화를 생성하기 위한 표준 라이브러리
프레임워크 (framework)	- 라이브러리의 집합 - 목적에 맞게 코드를 작성할 수 있는 기본 아키텍처이며, 필수 구성 요소를 제공한다.



[그림 6] 파이썬 관련 개념도

▶ 필요 패키지 설치 방법

- 주피터 노트북 내에서 패키지 및 모듈 설치하기

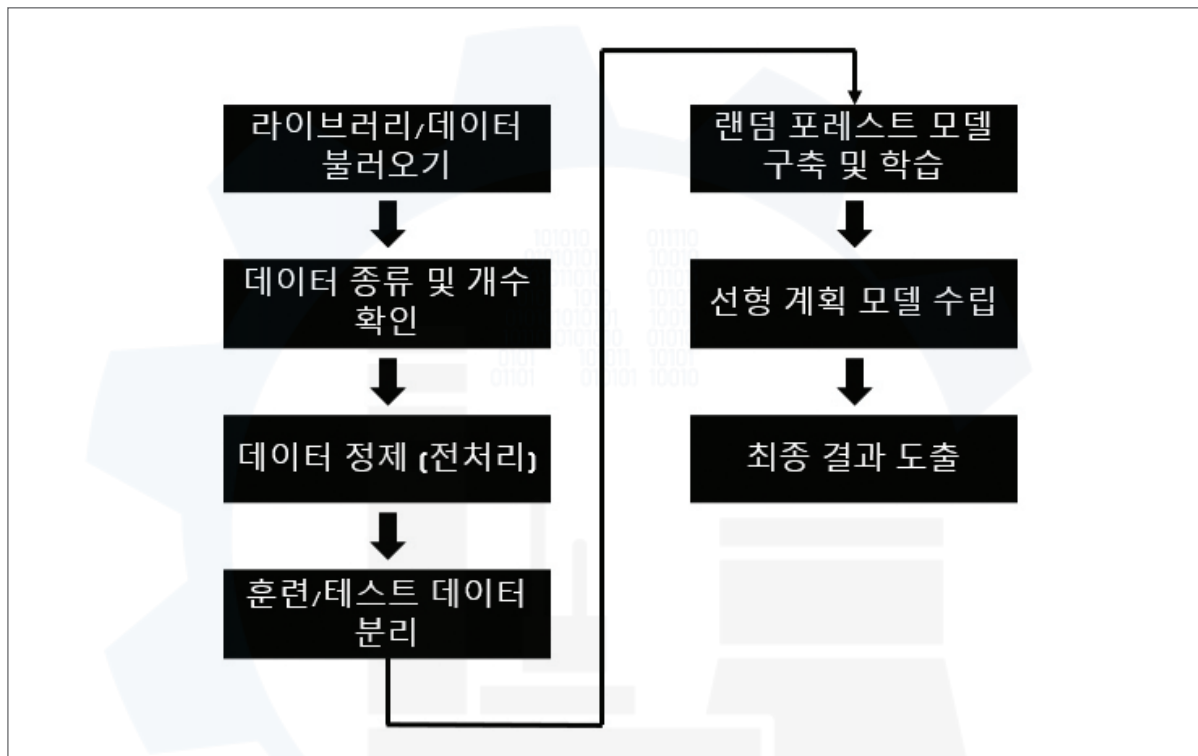
```
!pip install pandas
!pip install numpy
!pip install matplotlib
!pip install scikit-learn
!pip install missingno
!pip install datetime
!pip install seaborn
!pip install ortools
```

- `pip install` 패키지 및 모듈 이름은 설치를 뜻하며 영구적이다.
- `pip list`를 실행하면 현재 설치된 패키지 목록을 볼 수 있다. 직전에 설치한 'datetime'이 목록에 있는 것을 확인 가능하다.

pip list

In [6]:	pip list	
	cryptography	2.9.2
	cycler	0.10.0
	Cython	0.29.21
	cytoolz	0.10.1
	dask	2.20.0
	DateTime	4.3
	decorator	4.4.2
	defusedxml	0.6.0
	diff-match-patch	20200713
	distributed	2.20.0
	docutils	0.16
	matplotlib	3.3.0

2) 분석 단계별 프로세스 - Flow Chart



[그림 7] 지도 학습 적용을 위한 순환신경망 모델 구현 과정

[단계 ①] 라이브러리/데이터 불러오기

①-1. 라이브러리 불러오기

- 전체 실습 과정에 필요한 모든 패키지를 선언한다. 선언된 패키지에 포함된 클래스, 함수 등은 프로그램 내에서 자유롭게 이용(호출)할 수 있다.
- 패키지는 선언은 'import <패키지 이름>' 또는 'from <패키지 이름> import <모듈 이름>' 형식으로 선언한다.
- 패키지에 포함된 모든 모듈(객체, 함수)을 이용하고자 하는 경우 'import <패키지 이름>' 형식으로 선언하고, 패키지의 특정 모듈(들)만 이용하고자 하는 경우 'from <패키지 이름> import <모듈(들) 이름>' 형식으로 선언한다.

- 또한, 프로그램 코드 작성 시, 자주 사용하는 패키지의 경우 ‘import <패키지 이름> as <패키지 별칭>’ 형식으로 선언해서 패키지 이름 대신 별칭(alias)으로 대체해 간략하게 코드를 작성할 수 있다.

```
import os, random
import pandas as pd
import numpy as np
import seaborn as sns
import missingno as msno
import matplotlib.pyplot as plt
```

[코드 1] 라이브러리 불러오기

①-2. 데이터 불러오기

- 학습하고자 하는 데이터를 불러온다. csv 파일을 불러올 때에는 pandas의 ‘read_csv(‘파일 경로’)’ 기능을 사용한다. 랜덤 포레스트기반의 최대수요전력 예측 알고리즘 방식이므로 정수형 변수가 존재하는 데이터를 사용한다.
- ‘Resource_Management_Process.csv’의 파일을 읽어주며, 전처리 및 학습을 위해 제외할 컬럼명을 저장할 리스트를 정의한다.
- 일관된 코드의 결과 값을 위해 SEED에 2022값을 넣어서 랜덤성을 고정한다.

```
data_frame = pd.read_csv('./Resource_Management_Process.csv')
drop_col_list = []
SEED = 2022
random.seed(SEED)
```

[코드 2] 데이터 불러오기

[단계 ②] 데이터 종류 및 개수 확인

②-1. 데이터 종류 및 개수 확인 가이드

- 가장 기본적인 데이터 종류 및 개수를 확인하는 방법은 상기에서 csv 파일을 불러오기 위해 선언한 변수를 사용하는 방법이다.

data_frame

	Datetime	Production	Temperature	Humidity	Power_Cost	DoW	Worker_Power	Man_Cost	Power_Usage
0	2021-02-08 0:15	116	-4.2	65.6	109.8	Monday	0.13	1.5	103
1	2021-02-08 0:30	116	-4.2	63.5	109.8	Monday	0.13	1.5	100
2	2021-02-08 0:45	114	-4.2	67.6	109.8	Monday	0.14	1.5	109
3	2021-02-08 1:00	114	-5.5	72.8	109.8	Monday	0.13	1.5	161
4	2021-02-08 1:15	116	-5.5	67.4	109.8	Monday	0.15	1.5	136
...	...	...	...	...	...	...	...	...	...
24474	2021-10-21 22:45	129	23.6	74.0	167.2	Thursday	0.30	1.5	162
24475	2021-10-21 23:00	217	22.5	74.6	167.2	Thursday	1.07	1.5	134
24476	2021-10-21 23:15	217	22.2	71.3	167.2	Thursday	1.05	1.5	157
24477	2021-10-21 23:30	213	23.1	72.7	167.2	Thursday	1.05	1.5	163
24478	2021-10-21 23:45	215	22.2	71.6	167.2	Thursday	1.06	1.5	157

24479 rows × 9 columns

[코드 3] 데이터 호출



- 상기의 방법으로 총 해당 파일의 변수, 값, 행 및 열의 수 등을 확인할 수 있다.
- 상기 방법 외에도 어떤 데이터 종류가 있는지 간략히 확인할 수 있는 방법에는 'head()'를 사용한다. 기본으로 위에서 5개의 row를 읽어온다. 'head()'의 ()안에 읽어올 row의 수를 입력할 수 있다.

```
data_frame.head()
```

	Datetime	Production	Temperature	Humidity	Power_Cost	DoW	Worker_Power	Man_Cost	Power_Usage
0	2021-02-08 0:15	116	-4.2	65.6	109.8	Monday	0.13	1.5	103
1	2021-02-08 0:30	116	-4.2	63.5	109.8	Monday	0.13	1.5	100
2	2021-02-08 0:45	114	-4.2	67.6	109.8	Monday	0.14	1.5	109
3	2021-02-08 1:00	114	-5.5	72.8	109.8	Monday	0.13	1.5	161
4	2021-02-08 1:15	116	-5.5	67.4	109.8	Monday	0.15	1.5	136

[코드 4] head()를 활용한 기본 데이터 호출

- 'head()'와 반대로 뒤에서부터 데이터 구성을 확인할 수 있는 방법이 있다. 'tail()'을 사용하면 가능하며 기본으로 밑에서 5개의 row를 읽어온다. 'tail()'의 () 안에 읽어올 row의 수를 입력할 수 있다.

```
data_frame.tail()
```

	Datetime	Production	Temperature	Humidity	Power_Cost	DoW	Worker_Power	Man_Cost	Power_Usage
24474	2021-10-21 22:45	129	23.6	74.0	167.2	Thursday	0.30	1.5	162
24475	2021-10-21 23:00	217	22.5	74.6	167.2	Thursday	1.07	1.5	134
24476	2021-10-21 23:15	217	22.2	71.3	167.2	Thursday	1.05	1.5	157
24477	2021-10-21 23:30	213	23.1	72.7	167.2	Thursday	1.05	1.5	163
24478	2021-10-21 23:45	215	22.2	71.6	167.2	Thursday	1.06	1.5	157

[코드 5] tail()을 활용한 기본 데이터 호출

- 'info()'를 사용하여, 전체적인 데이터의 구조 및 타입을 확인할 수 있다.

```
data_frame.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 24479 entries, 20371 to 20476
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Datetime        24479 non-null object
1   Production       24479 non-null int64
2   Temperature     24479 non-null float64
3   Humidity        24479 non-null float64
4   Power_Cost      24479 non-null float64
5   DoW             24479 non-null object
6   Worker_Power    24479 non-null float64
7   Man_Cost        24479 non-null float64
8   Power_Usage     24479 non-null int64
9   season          24479 non-null int64
10  day             24479 non-null int64
dtypes: float64(5), int64(4), object(2)
memory usage: 2.2+ MB
```

[코드 6] info()를 사용한 csv의 전체 데이터 구성 확인



[단계 ③] 데이터 정제 (전처리)

③-1. NULL값 처리

- 본 실습에서 사용되는 데이터에는 불필요한 데이터가 존재하지 않기 때문에 해당 과정은 생략하지만, 있을 경우를 대비하여 null값의 존재여부를 확인하는 코드를 작성한다.
- 데이터에 존재하는 null값을 찾기 위해서는 먼저 다음과 같이 어떤 변수에 몇 개의 변수에 결측치가 존재하는지 확인할 수 있다.
- 'isnull()'을 사용할 경우, null값에는 True, 그 외에는 False가 표현된다. 직관적으로 확인이 가능하지만, 해당 변수의 몇 개의 결측치가 존재하는지 파악하기 위해서는 추가로 'sum()'을 사용하여야 한다.
- 이를 응용한 다음의 코드로 null값을 퍼센트로 확인한다.

```
for col in data_frame.columns:
    null_count = column: {>10}\t Percent of NaN value: {:.2f}%'.format(col, 100 * (data_frame[col].isnull().sum() /
data_frame[col].shape[0]))
    print(null_count)
```

column: Datetime	Percent of NaN value: 0.00%
column: Production	Percent of NaN value: 0.00%
column: Temperature	Percent of NaN value: 0.00%
column: Humidity	Percent of NaN value: 0.00%
column: Power_Cost	Percent of NaN value: 0.00%
column: DoW	Percent of NaN value: 0.00%
column: Worker_Power	Percent of NaN value: 0.00%
column: Man_Cost	Percent of NaN value: 0.00%
column: Power_Usage	Percent of NaN value: 0.00%
column: season	Percent of NaN value: 0.00%
column: day	Percent of NaN value: 0.00%

[코드 7] isnull().sum()을 활용한 결측치 개수 확인

- 학습을 위해서 결측치가 존재하는 데이터를 사용할 수는 없다. 다행히 상기의 데이터는 결측치가 존재하지 않음을 확인할 수 있다.

③-2. 데이터 특성 파악

- 지금까지 각 변수의 데이터 값에 대한 결측치를 제거하였다. 결측치가 존재하지 않기에 지금부터는 정확한 수치형 데이터들의 요약통계량을 확인할 수 있다. 이 경우, 'describe()' 함수를 사용한다.

```
data_frame.describe()
```

	Production	Temperature	Humidity	Power_Cost	Worker_Power	Man_Cost	Power_Usage
count	24479.000000	24479.000000	24479.000000	24479.000000	24479.000000	24479.000000	24479.000000
mean	231.557988	16.332203	63.231562	162.533135	1.042010	1.312492	136.468238
std	214.886001	9.679980	24.224441	30.831938	1.989461	0.242068	58.113685
min	112.000000	-13.700000	-2.000000	109.800000	0.120000	1.000000	39.000000
25%	114.000000	9.500000	45.600000	167.200000	0.140000	1.000000	68.000000
50%	126.000000	17.800000	67.400000	167.200000	0.250000	1.500000	147.000000
75%	276.000000	24.100000	84.900000	191.600000	1.310000	1.500000	186.000000
max	2573.000000	36.000000	97.000000	191.600000	48.530000	1.500000	270.000000

[코드 8] 데이터 통계량 확인

- 해당 함수를 활용하면 상기와 같이 데이터의 개수, 평균, 표준편차, 최소값, 최대값 등을 확인할 수 있다.
- 해당 수치형 데이터들의 상관관계 또한 확인할 수 있다. 'corr()' 함수를 사용하여 다음의 코드로 각 데이터 간 상관관계를 파악할 수 있다.

```
corr = data_frame.corr()
corr.style.background_gradient(cmap='coolwarm')
```

	Production	Temperature	Humidity	Power_Cost	Worker_Power	Man_Cost	Power_Usage
Production	1.000000	0.123660	-0.106838	0.073177	0.784709	-0.278331	0.516299
Temperature	0.123660	1.000000	0.398863	0.807716	0.035426	-0.192990	0.044743
Humidity	-0.106838	0.398863	1.000000	0.521753	-0.119009	0.305062	-0.089923
Power_Cost	0.073177	0.807716	0.521753	1.000000	-0.017753	0.000054	0.052568
Worker_Power	0.784709	0.035426	-0.119009	-0.017753	1.000000	-0.181563	0.289897
Man_Cost	-0.278331	-0.192990	0.305062	0.000054	-0.181563	1.000000	-0.219438
Power_Usage	0.516299	0.044743	-0.089923	0.052568	0.289897	-0.219438	1.000000

[코드 9] corr()를 활용한 상관관계 확인

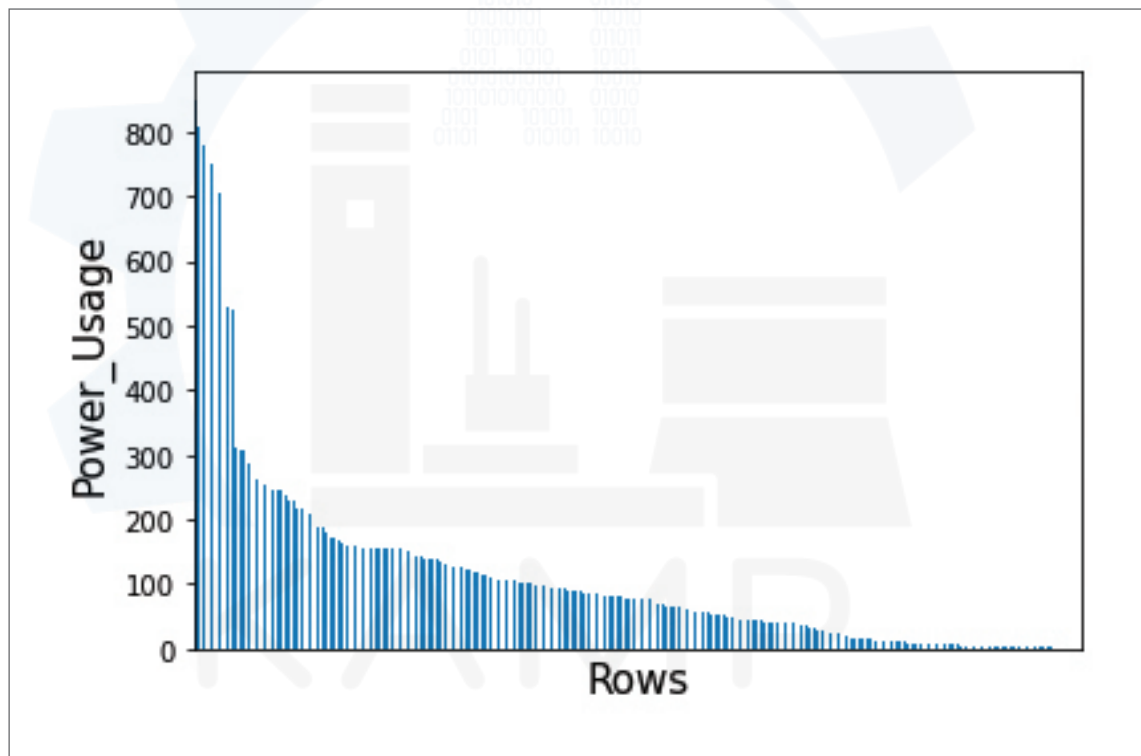
- 상관관계 값의 경우, -1과 1 사이에 존재한다. 1에 가까울수록 양의 상관관계가 높으며 -1에 가까울수록 음의 상관관계가 높다고 볼 수 있다. 대략적인 상관관계수의 의미는 다음과 같다.
 - 0.0 ~ 0.2 : 상관관계가 거의 없다.
 - 0.2 ~ 0.4 : 상관관계가 낮다.
 - 0.4 ~ 0.6 : 상관관계가 있다.
 - 0.6 ~ 0.8 : 상관관계가 높다.
 - 0.8 ~ 1.0 : 상관관계가 매우 높다.

- 각 데이터 간의 상관관계를 확인하였을 때, 서로 이어진 시간의 흐름끼리 상관관계가 매우 높다고 볼 수 있다. (0.9와 1 사이의 값). 또한, 추가로 생산량이 상관관계가 있다고 볼 수 있다. (0.5 정도의 값)

③-3. 데이터 정제

- 본 실습에서 모델의 학습 및 결과 도출에 사용될 레이블에 대한 특성을 파악한다. pandas의 'value_counts()'와 'plot.bar()'를 이용하여 레이블의 전체적인 경향성을 파악한다.

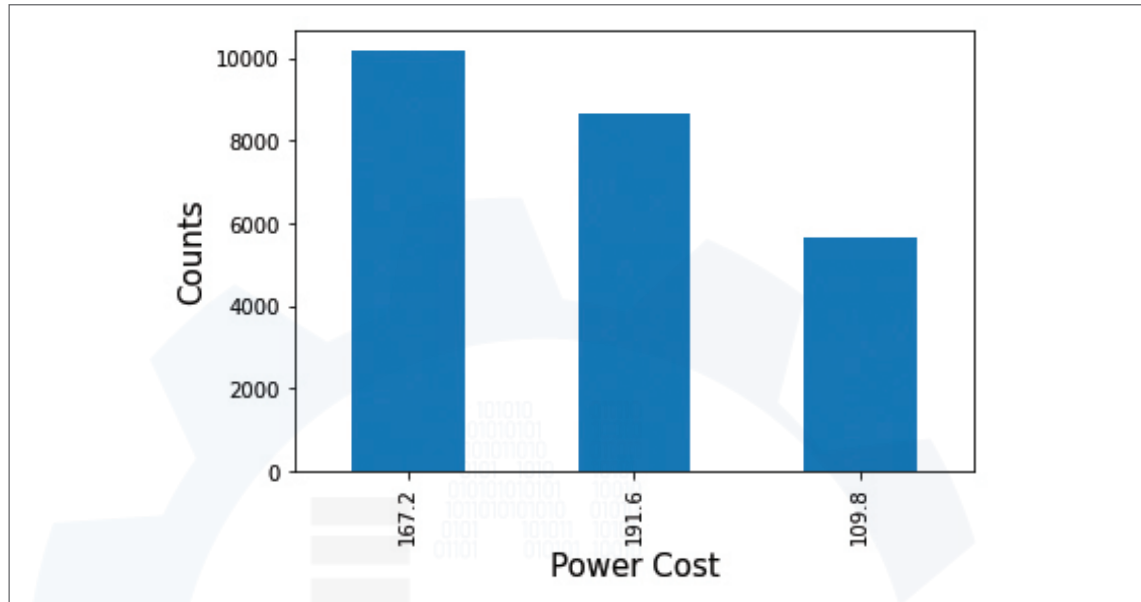
```
ax = plt.gca()
data_frame['Power_Usage'].value_counts().plot.bar(ax=ax)
ax.set_xlabel('Rows',fontsize=15)
ax.set_ylabel('Power_Usage',fontsize=15)
ax.set_xticks([])
ax.axes.xaxis.set_visible(True)
```



[코드 10] 설비 에너지들의 효율정도에 대한 경향성 확인

- 본 실습에서 'Power_Usage' 변수는 에너지들의 효율정도를 나타낸다. 위 코드는 스케줄에 포함된 분포도를 출력한 것이다.

```
ax = plt.gca()
data_frame['Power_Cost'].value_counts().plot.bar(ax=ax)
ax.set_xlabel('Power Cost',fontsize=15)
ax.set_ylabel('Counts',fontsize=15)
```



[코드 11] 전력 비용에 대한 분포도 확인

- 본 실습에서 'Power_Cost' 변수는 에너지 효율정도에 대한 요금을 나타낸다. 이를 활용하여, 각 요일별 설비 에너지들의 효율성에 따른 요금의 종류를 출력한다.

```
drop_col_list.append('Power_Cost')
```

- 추후 선형계획 모델에 사용될 변수 'Power_Cost'를 따로 drop_col_list에 미리 추가한다.

```
day_list = list(data_frame['DoW'].unique())
print("DoW의 unique 값 : ",day_list)
```

```
DoW의 unique 값 : ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
```

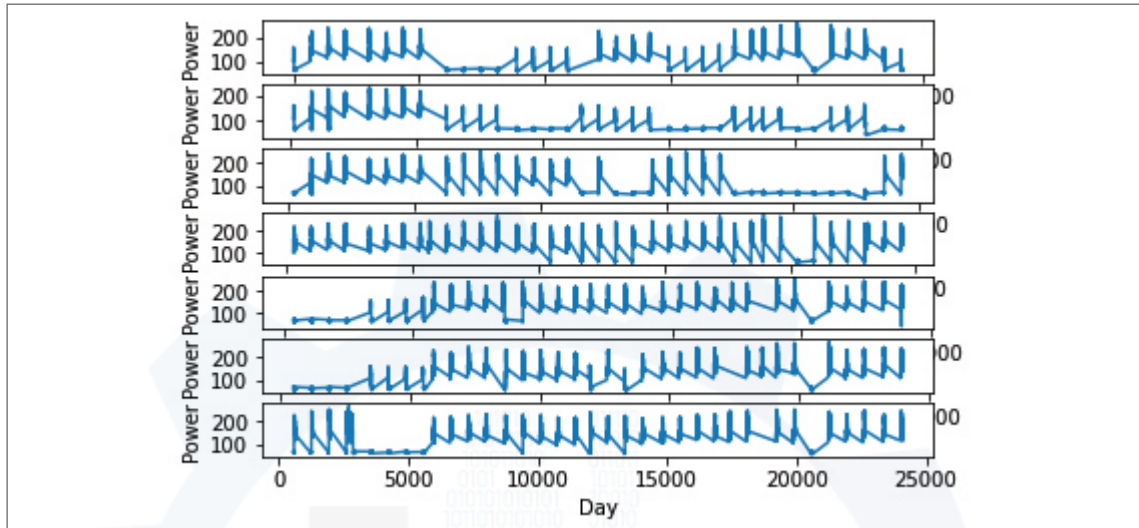
[코드 12] 공정 스케줄 운영 요일확인

- 본 실습에서 'DoW' 변수는 요일을 나타낸다. 이를 활용하여, 각 요일별 설비 에너지들의 효율정도를 출력한다.


```

day_list = list(data_frame['DoW'].unique())

for di,day in enumerate(day_list):
    plt.subplot(7,1,di+1)
    plt.xlabel('Day')
    plt.ylabel('Power')
    data_frame[data_frame['DoW']==day]['Power_Usage'].plot()
  
```



[코드 13] 요일별 설비에너지들의 분포도 확인

- 본 실습에 날짜를 나타내는 Date타입의 'Datetime'이 존재한다. 이 컬럼으로부터 요일과 계절 값을 추출하여 새로운 컬럼 'season'과 'day'를 추가적으로 생성하는 함수를 정의한다. 계절의 경우 '3,4,5'월의 경우 봄, '6,7,8'은 여름, '9,10,11'은 가을 그리고 나머지 달들은 겨울로써 취급한다. 각각 봄: 0, 여름: 1, 가을: 2, 겨울: 3의 값으로 설정한다. 전처리 후 기존에 정의한 'drop_col_list'에 'Datetime'을 추가한다.

```

def categorical_season_label(data_frame):
    date_list = [int(datestr.split(' ')[0].split('-')[1]) for datestr in list(data_frame['Datetime'])]
    season_list = []
    for date in date_list:
        if 3<= date and date <6:
            season_list.append(0)
        elif 6<=date and date <9:
            season_list.append(1)
        elif 9<=date and date <12:
            season_list.append(2)
        else:
            season_list.append(3)
    return season_list

data_frame['season'] = categorical_season_label(data_frame)
drop_col_list.append('Datetime')
  
```

[코드 14] 계절 추출 함수

- 요일의 경우 'Datetime'에서 각 데이터들이 가지는 요일들을 추출하여 각각의 요일에 대한 값들을 랜덤 포레스트 모델의 학습을 위해 월: 0, 화: 1, 수: 2, 목: 3, 금: 4, 토: 5, 일: 6로 대체한다. 전처리 후 기존에 정의한 'drop_col_list'에 'DoW'를 추가한다.

```
def categorical_Day_label(data_frame):
    day_list = list(data_frame['DoW'].unique())
    day_dict = {}
    for i, day in enumerate(day_list):
        day_dict[day] = i
    day_index_list = []
    for day_ in list(data_frame['DoW']):
        day_index_list.append(day_dict[day_])

    return day_index_list

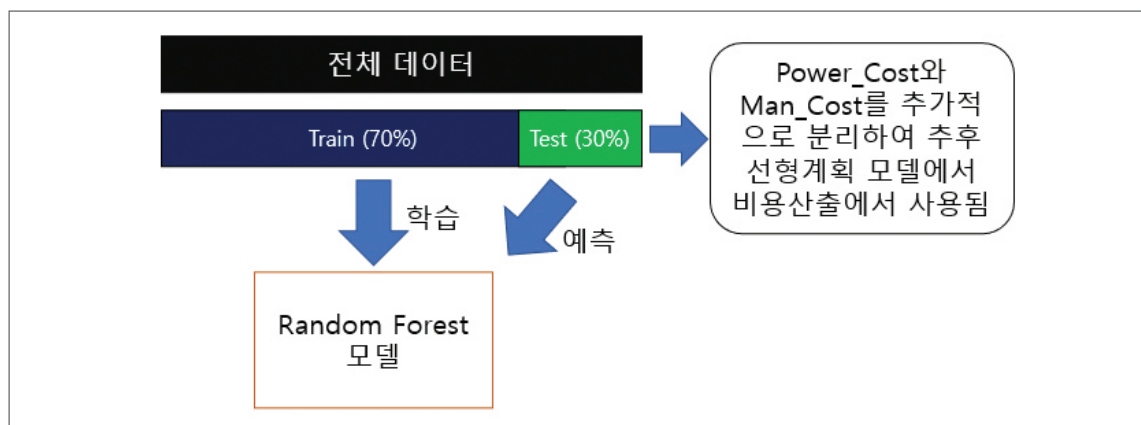
data_frame['day'] = categorical_Day_label(data_frame)
drop_col_list.append('DoW')
```

[코드 15] 요일 전처리 함수

[단계 ④] 훈련/테스트 데이터 분리

④-1. 훈련/테스트 데이터 분리 및 실습

- 랜덤 포레스트 모델에 데이터를 학습하고, 선형계획 모델 수립을 위해 데이터를 적절히 분리하는 작업이 필요하다. 파이썬에서 제공하는 라이브러리를 활용하여 훈련 및 테스트 데이터를 분리한다.
- 본 실습에서 'Power_Usage'를 예측하는 랜덤 포레스트 모델을 생성하고, 'Man_Cost'를 이용하여 비용을 최소화시키기 위한 답을 도출하는 선형계획 모델을 생성한다.
- 'Power_Cost'와 'Man_Cost'의 경우 랜덤 포레스트 모델이 예측한 결과 값을 이용하여 선형계획 모델에서 자원 소모 비용을 계산하는 수식의 계수로 쓰이기 때문에, test index에 맞추어 추가적으로 따로 분리한다.
- 본 실습에서는 모델 학습을 위해 전체 데이터의 70%를 훈련 데이터로 사용하고, 학습된 모델의 검증을 위해 전체 데이터의 30%를 테스트 데이터로써 사용한다.



[그림 8] 전체 데이터의 학습 & 테스트 데이터로 분리



- 레이블로 사용될 'Power_Usage'와 선형계획 모델의 변수로 사용될 'Man_Cost'는 모델 학습에 사용되지 않으므로, 따로 'drop_col_list'에 추가한다.

```
data_frame = data_frame.sample(frac=1)
train_len = round(len(data_frame)*0.7)
train_dataframe = data_frame[:train_len]
test_dataframe = data_frame[train_len:]

train_Power_usage=list(train_dataframe['Power_Usage'])
test_Power_usage=list(test_dataframe['Power_Usage'])

train_man_cost = list(train_dataframe['Man_Cost'])
train_power_cost = list(train_dataframe['Power_Cost'])
test_man_cost = list(test_dataframe['Man_Cost'])
test_power_cost = list(test_dataframe['Power_Cost'])

drop_col_list.append('Man_Cost')
drop_col_list.append('Power_Usage')
```

[코드 16] 학습 데이터 분리

[단계 ⑤] 랜덤 포레스트 모델 구축 및 학습

⑤-1. 랜덤 포레스트 모델 적용 후 학습

- 학습을 위한 랜덤 포레스트 모델과 학습 결과를 검증할 오차를 계산하는 함수들을 import한다. 훈련 데이터와 테스트 데이터에서 현재까지 'drop_col_list'에 추가한 컬럼들을 'drop'함수를 사용하여 제거한다. 그 후 'Power_Usage'를 이용하여 랜덤 포레스트 모델을 정의하고 학습한다.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error

test_data_x = test_dataframe.drop(drop_col_list,axis=1)
train_data_x = train_dataframe.drop(drop_col_list,axis=1)

rf_reg = RandomForestRegressor(max_depth=5)
rf_reg.fit(train_data_x,train_Power_usage)
```

[코드 17] 랜덤 포레스트 모델 정의 및 학습

- 학습 후 'mean_squared_error' 함수를 사용하여 손실값을 확인한다. Mean squared error는 다음 그림처럼 관측값과 예측값의 차이의 제곱합에 대한 평균을 나타내는 것이다. 관측값과 예측값의 차이인 오차의 제곱에 비례해서 손실 함수로부터 계산되는 손실값이 커진다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

[그림 9] mean squared error 수식



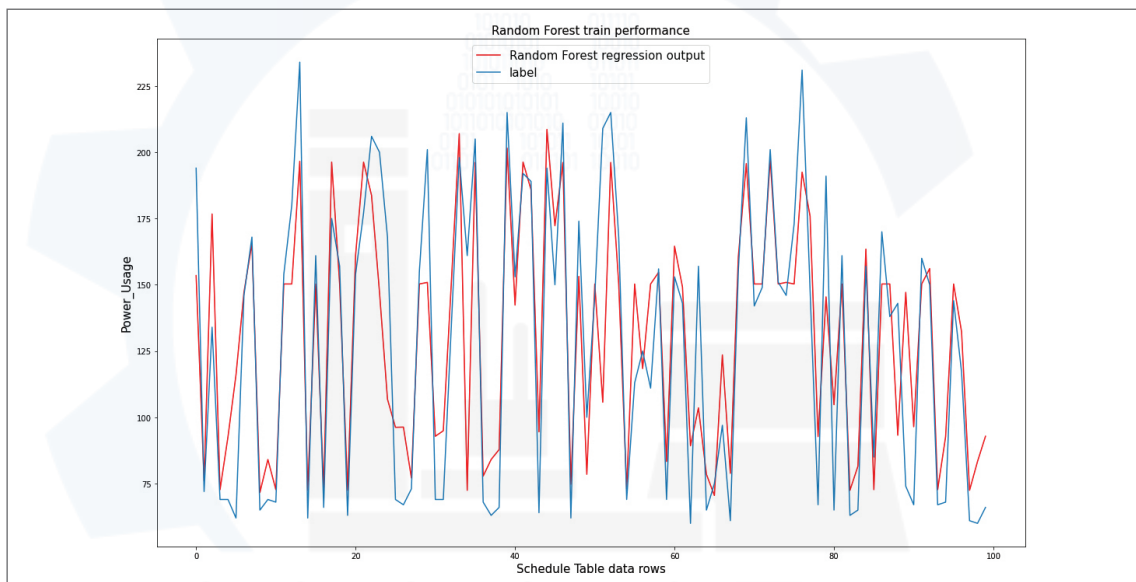
```
train_result = rf_reg.predict(train_data_x)
print(mean_squared_error(train_result, train_Power_usage))
```

861.3502156137984

[코드 18] 손실값 확인

- 학습된 모델의 결과를 훈련데이터에 대한 결과값에 대해 그래프로 출력한다.

```
fig, ax = plt.subplots(figsize=(20,12))
ax.plot(train_result[:100], color='red', label='Random Forest regression output')
ax.plot(train_Power_usage[:100], label='label')
ax.set_xlabel('Schedule Table data rows')
ax.set_ylabel('Power_Usage')
ax.set_title('Random Forest train performance')
ax.xaxis.label.set_fontsize(15)
ax.yaxis.label.set_fontsize(15)
ax.title.set_fontsize(15)
plt.legend(fontsize=15)
```



[코드 19] 훈련 데이터에 대한 학습 성능 확인

- 학습된 모델에 대한 성능 결과를 테스트 데이터셋에 적용하여 결과를 확인한다. 학습 데이터로 훈련한 랜덤 포레스트 모델의 테스트 데이터셋에서 성능을 'mean_squared_error'로 확인한다.

아래 실습에서 사용되는 mean squared error는 스케줄 데이터로 학습한 랜덤 포레스트의 예측값과 Power_usage의 모든 차이 값에 대한 평균 제곱 오차의 합을 나타낸다.

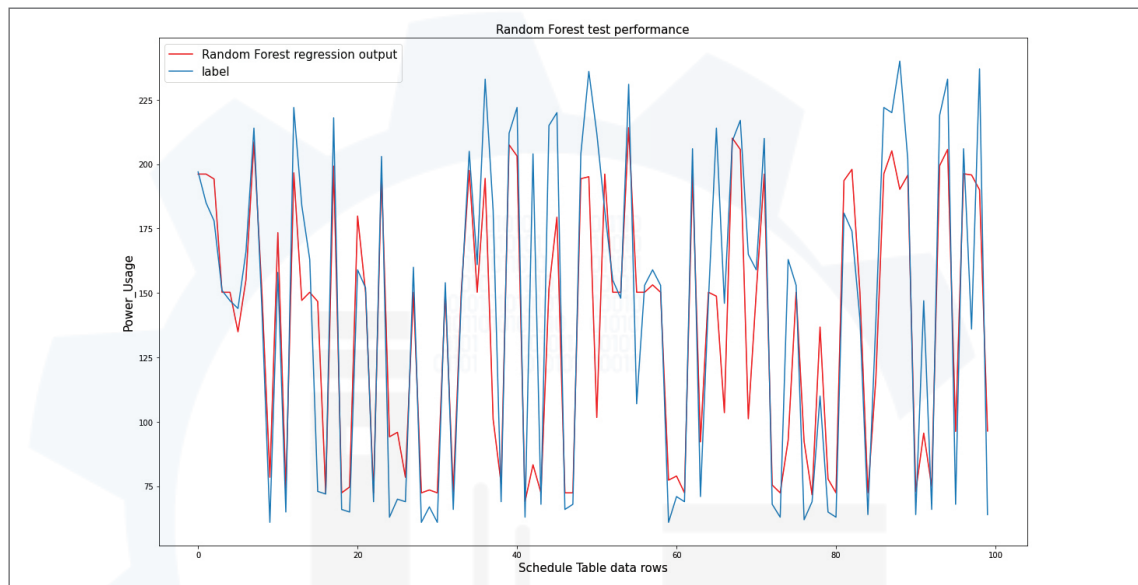
```
test_result = rf_reg.predict(test_data_x)
print(mean_squared_error(test_result, test_Power_usage))
```

895.4681314584349

[코드 20] 랜덤 포레스트 학습 성능 확인

- 학습된 모델에 대한 성능 결과를 테스트 데이터셋에 적용하여 결과를 확인한다. 경향성 확인을 위해 100개를 샘플로써 확인한다.

```
fig, ax = plt.subplots(figsize=(20,12))
ax.plot(test_result[:100],color='red',label='Random Forest regression output')
ax.plot(test_Power_usage[:100],label='label')
ax.set_xlabel('Schedule Table data rows')
ax.set_ylabel('Power_Usage')
ax.set_title('Random Forest test performance')
ax.xaxis.label.set_fontsize(15)
ax.yaxis.label.set_fontsize(15)
ax.title.set_fontsize(15)
plt.legend(fontsize=15)
```

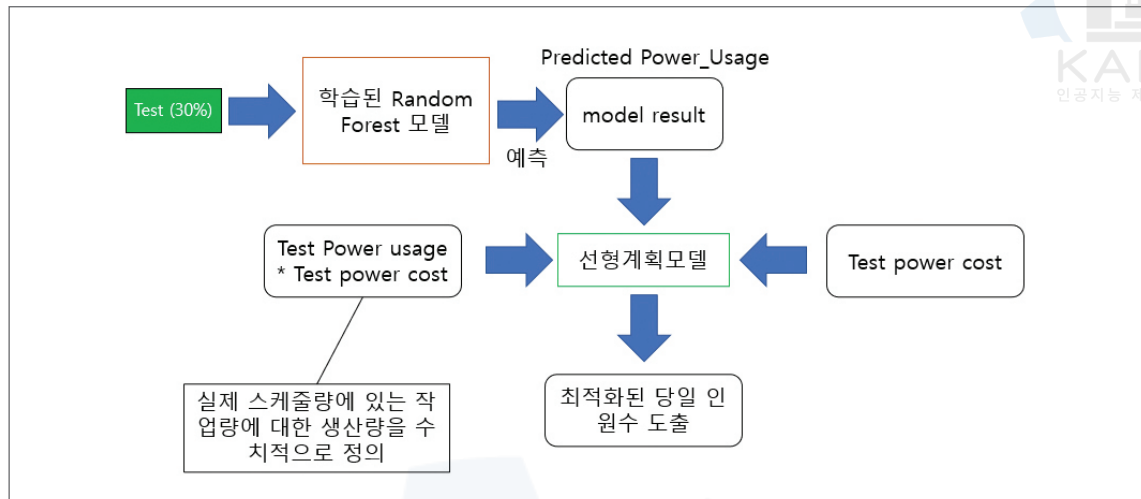


[코드 21] 테스트 데이터에 대한 학습된 모델 성능 확인

[단계 ⑥] 선형 계획 모델 수립

⑥-1. 선형 계획 모델 적용

- 학습된 모델의 결과 값을 사용하여 선형계획 모델을 수립한다. 이를 위해 ortools 라이브러리를 import한다.
- 'Power_Usage'을 레이블로 학습한 랜덤 포레스트 모델의 학습 결과와 'Power_Cost'와 'Man_Cost'를 활용하여 당일 필요한 인원의 명수를 정의하는 선형계획 모델을 구축한다. 즉, 수립한 선형계획 모델이 도출하는 결과 값은 목적 생산량을 달성하기 위한 생산 공정인원을 몇 명 투입해야 하는지를 구하는 것이다.



[그림 10] 최적인원 도출을 위한 선형계획모델 도출

- 당일 목적 생산량의 경우 실제 공정 설비 에너지 효율정과 전력비용을 곱한 것을 정량적인 실제 스케줄의 생산량으로 정의하여 사용한다. 학습된 랜덤 포레스트가 예측한 Power Usage와 Test power cost를 이용하여 당일 목적 생산량을 달성하기 위한 최적 인원수 X값을 도출한다. 최소 인원은 10명 이상으로 설정한다.

$$\text{Test Power usage} * \text{Test power cost} \leq X * (\text{Test power cost} + \text{model result})$$

$$10 \leq X$$

[그림 11] 최적 인원 투입 정의를 위한 선형계획 모델 수립

- 아래 코드는 위 선형 계획 모델을 수립하고, 최적인원을 도출하는 것을 구현한 것이다.
- 도출된 생산인원은 'solution_list'에 저장한다. 'work_list'와 'sol_work_list'의 경우 추후 학습된 모델의 출력 값에 대한 검증 시각화를 위해 사용될 리스트이다.

```

from ortools.linear_solver import pywraplp

solution_list = []
work_list = []
sol_work_list = []

for i, cost in enumerate(test_man_cost):
    solver = pywraplp.Solver.CreateSolver("GLOP")
    x = solver.IntVar(10, solver.infinity(), "x")
    solver.Minimize(x*(cost+test_result[i]))
    solver.Add(x*(cost+test_result[i])>=test_power_cost[i]*test_Power_usage[i])
    solver.Add(x>=10)
    solver.Solve()
    solution_list.append(x.solution_value())
    work_list.append(test_power_cost[i]*test_Power_usage[i])
    sol_work_list.append(int(x.solution_value()+1)*(cost+test_result[i]))
  
```

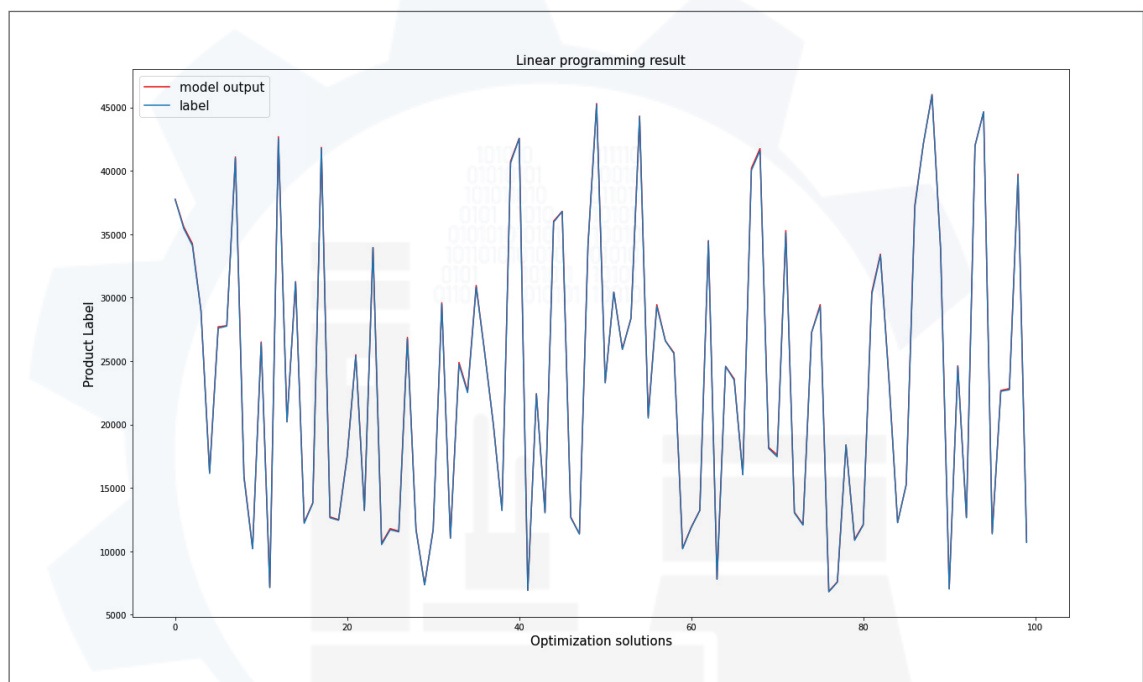
[코드 22] 산출된 노동력에 대한 인원수 변환

[단계 ⑦] 최종 결과 도출

⑦-1. 결과 시각화

- 예측한 결과값을 그래프로 출력하여 확인한다.

```
fig, ax = plt.subplots(figsize=(20,12))
ax.plot(sol_work_list[:100],color='red',label='model output')
ax.plot(work_list[:100],label='label')
ax.set_xlabel('Optimization solutions')
ax.set_ylabel('Product Label')
ax.set_title('Linear programming result')
ax.xaxis.label.set_fontsize(15)
ax.yaxis.label.set_fontsize(15)
ax.title.set_fontsize(15)
plt.legend(fontsize=15)
```



[코드 23] 최적 인원 투입 정의를 위한 선형계획 모델 수립출력 값 확인

- 자원 최적화를 위한 목적함수를 정의하고 정의된 목적함수와 제약조건 간의 관계를 그래프로 표현하였고, 생산량을 달성하기 위한 최소 공정 생산인원을 도출하는 모델을 수립하였다.



3. 유사 타 현장의 「표면처리 자원최적화 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 제조 공장 뿐만 아니라 24시간 운영해야 하는 모든 산업 현장에 적용할 수 있다. 24시간 운영되는 산업 현장에서 생산량에 따른 전력 소비 데이터를 기록 및 수집이 쉽지는 않다.
- 생산에 소비되는 전력 소비 데이터의 패턴이 추출할 수 있을 만큼의 데이터가 수집이 된다면, 본 가이드북에서 구축한 랜덤 포레스트 회귀 모델을 사용한 분석을 통하여, 효율적인 생산 계획을 구축할 수 있고 이를 통하여 생산 비용을 최소한으로 줄일 수 있다.
- 제조 공정 내에서 환경에 대한 데이터들에 대해 통계적인 기법을 적용하여 새로운 인사이트를 도출할 수 있다면, 좀 더 효과적인 솔루션을 제안할 수 있다. 예를 들어, 공정 내의 근무자들의 이동경로 패턴을 추출할 수 있다면, 근무자들의 편의성과 생산의 효율성을 극대화하기 위해 제조 물품의 위치, 각 제조 물품별 공정 내의 이동 경로 도출 등의 솔루션을 제안할 수 있다.
- 도금업 특성상 수주 제품의 종류가 다양해짐으로 인하여 제품의 형태와 표면적, 질량 등 다양한 요인에 따라 달라지는 공정운영 설정을 결정하기 어려워진다. 설비 에너지 효율 정도와 전력 비용 등 주요 공정변수에 대한 가이드라인을 제공할 수 있다.

3.2 본 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 상기 데이터셋의 구조를 그대로 적용하기보다는 각 공정에 맞는 데이터를 수집하여 해당 모델을 적용해야 한다. 예를 들어, 정밀한 표면처리 공정 환경 구축을 위해 제조 공정 내의 기기 센서의 위치를 활용한 데이터, 공정 환경에 영향을 미칠 수 있는 공정 내의 온도, 습도 등의 연관성이 있는 데이터들 위주로 수집한다.
- 학습된 모델의 예측값을 사용하여 해당 모델을 적용하기 전에, 관련된 데이터가 무엇인지 정의를 하고 그에 맞는 데이터를 수집해야 한다.
- 구축하고자 하는 모델의 실용성을 보장하기 위해, 현장에 근무 중인 근무자들과의 소통을 지속적으로 해야 하고, 도출된 모델의 솔루션에 대한 실효성을 파악해야 한다.
- Worker_Power 같은 인력에 대한 데이터는 일반적으로 수집할 수 있는 데이터와 달리 주관적인 입장이 들어갈 수 밖에 없기 때문에 데이터에 대한 신뢰도를 체크하면서 수집해야 한다.
- 본 분석에서 활용한 값은 다른 제조 현장과 맞지 않을 수 있기에 현장 전문가의 의견을 반영하여 적용 여부를 결정해야 한다.

3 부록_분석환경 구축을 위한 설치 가이드

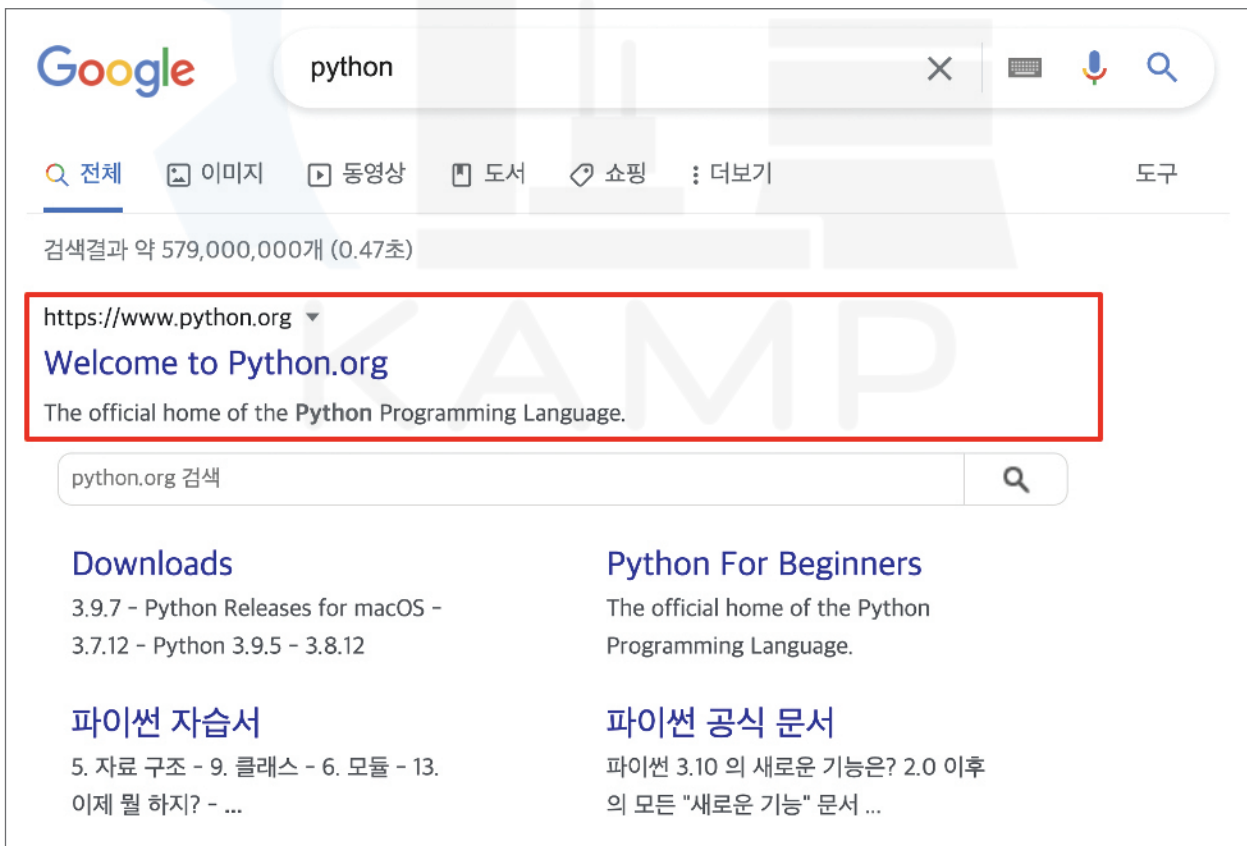
1. 파이썬(Python) 설치

파이썬은 컴퓨터 프로그래밍 언어로서, 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하는 방법을 안내한다.

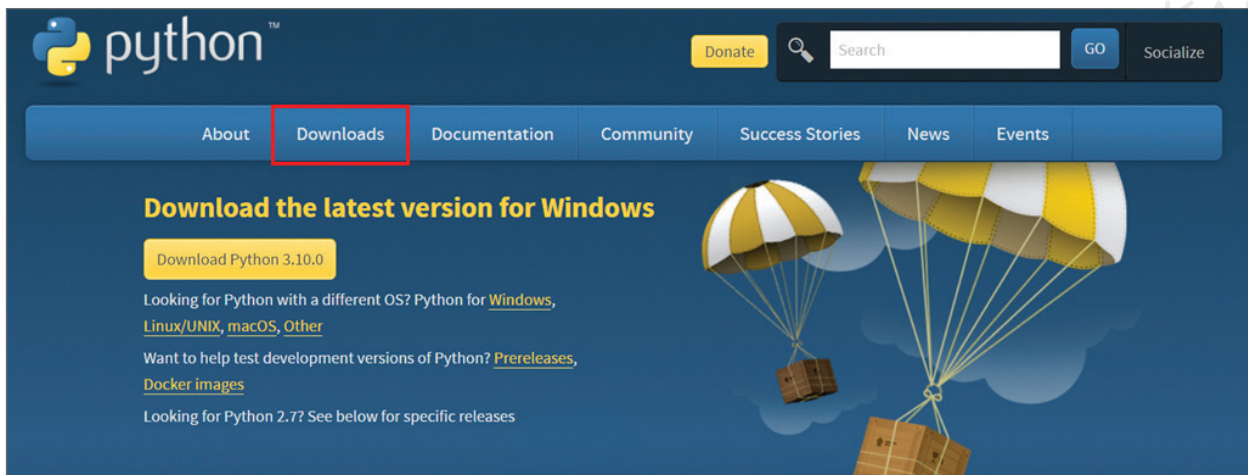
① google.com 등의 검색 엔진에 'python'을 검색한다.



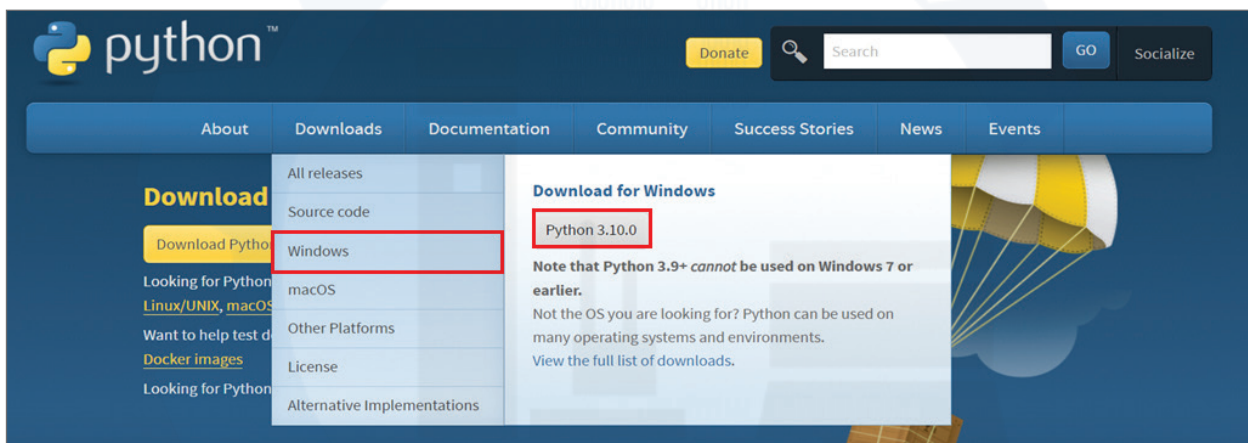
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



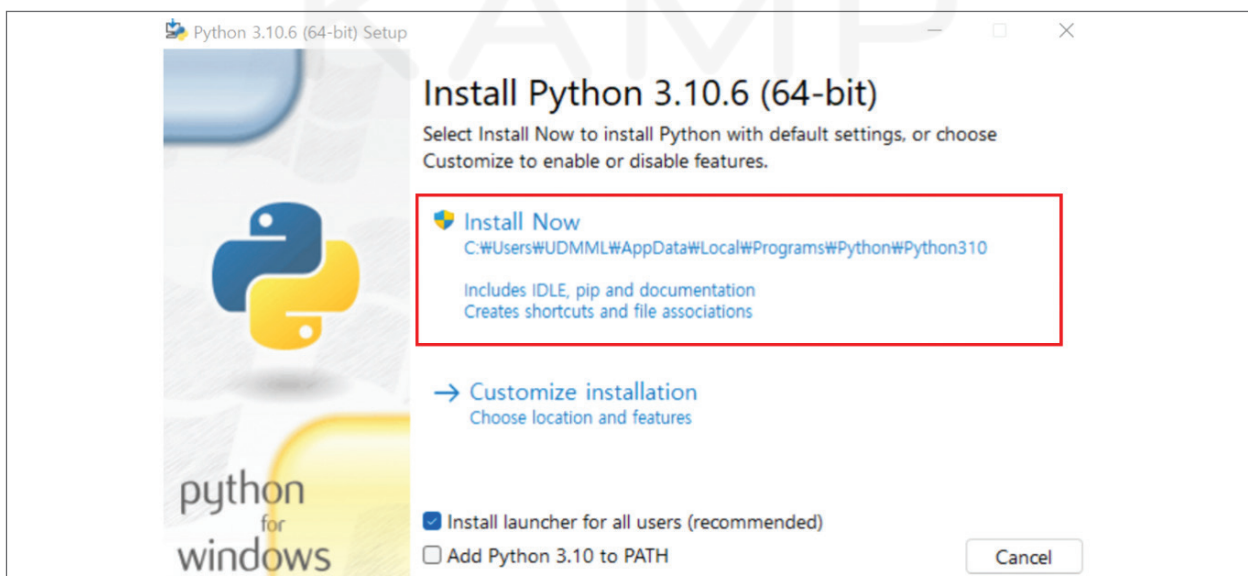
③ 클릭 후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



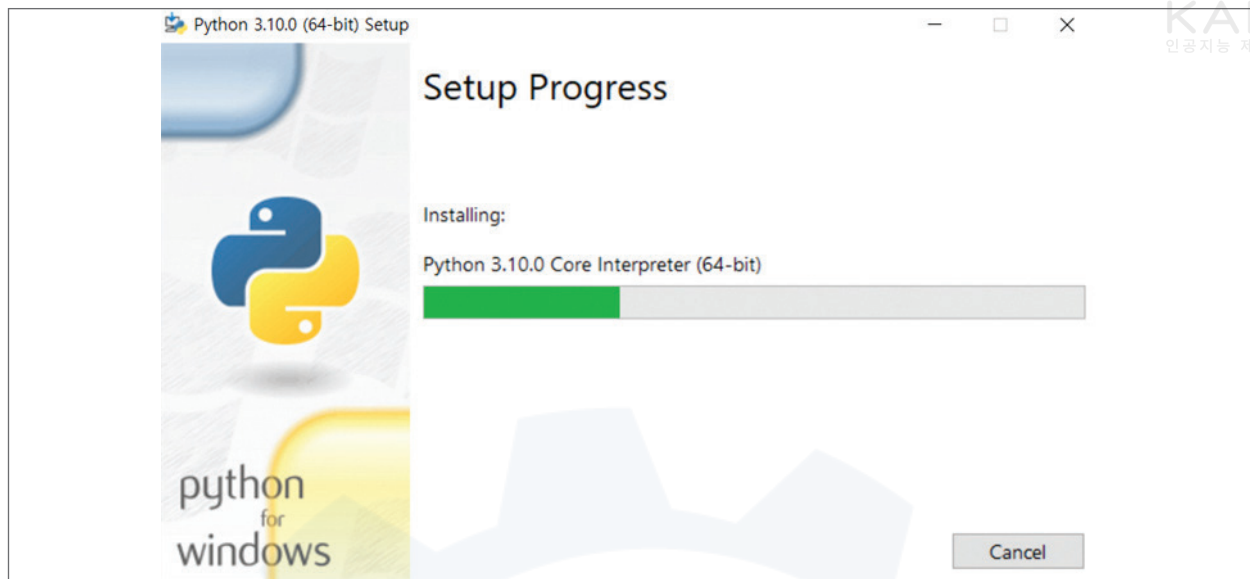
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 버전은 업데이트 될 수 있다.)



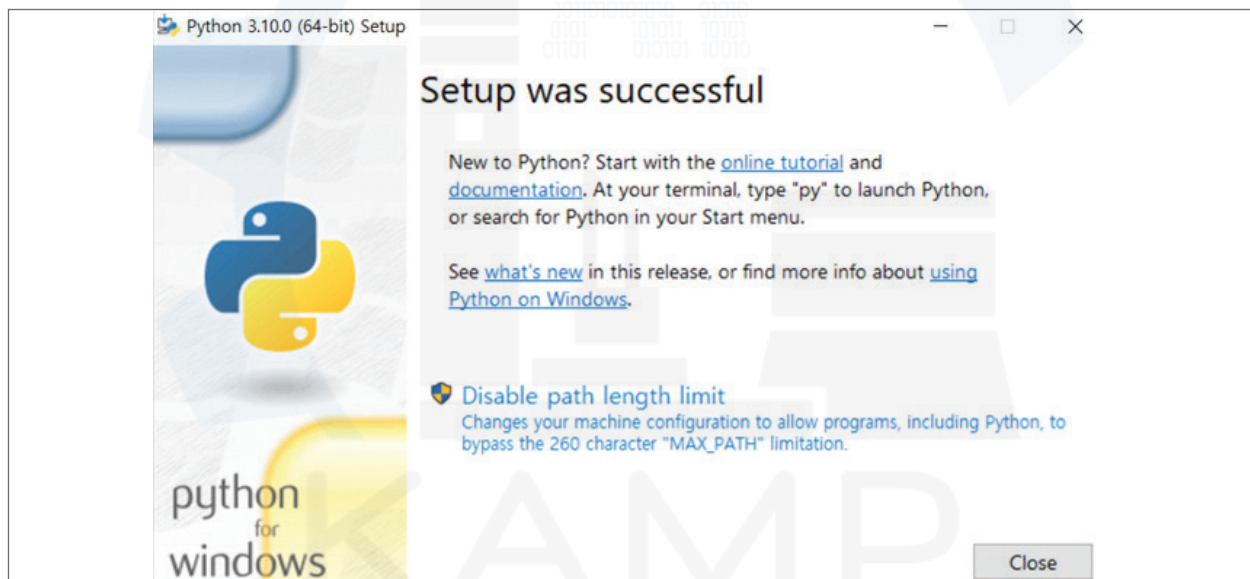
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



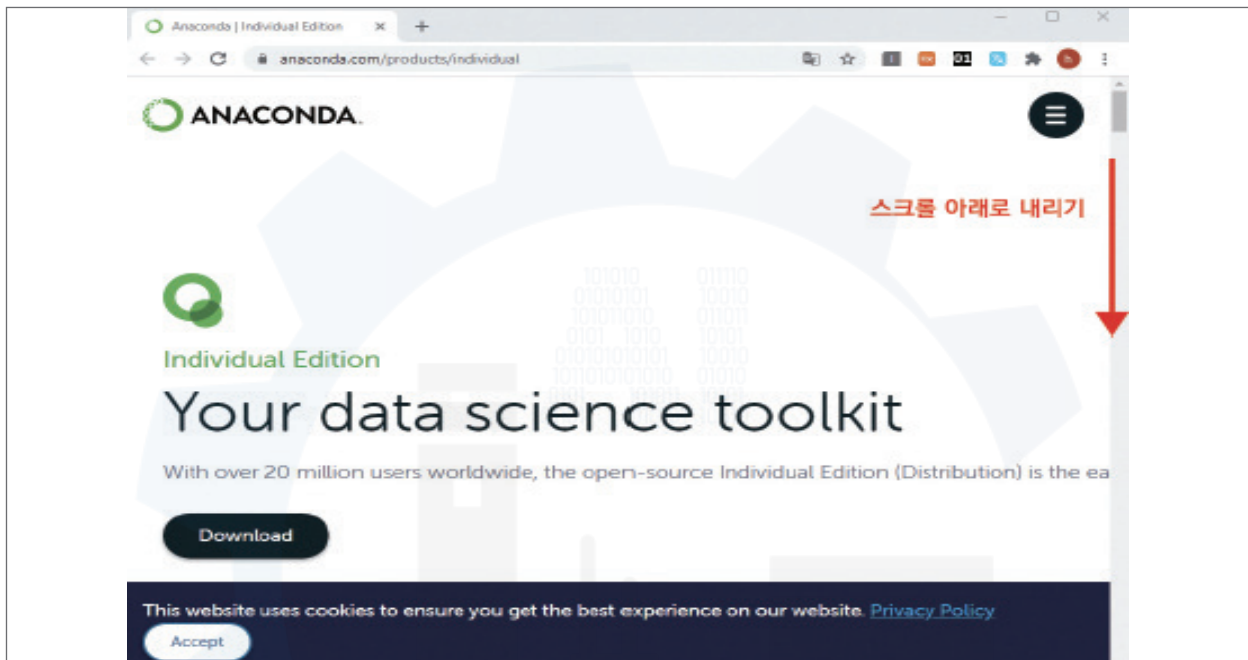
⑦ 아래와 같은 창이 나타나면 설치 완료된 것으로, 확인 후 종료한다. [설치완료]



2. 아나콘다(Anaconda) 설치

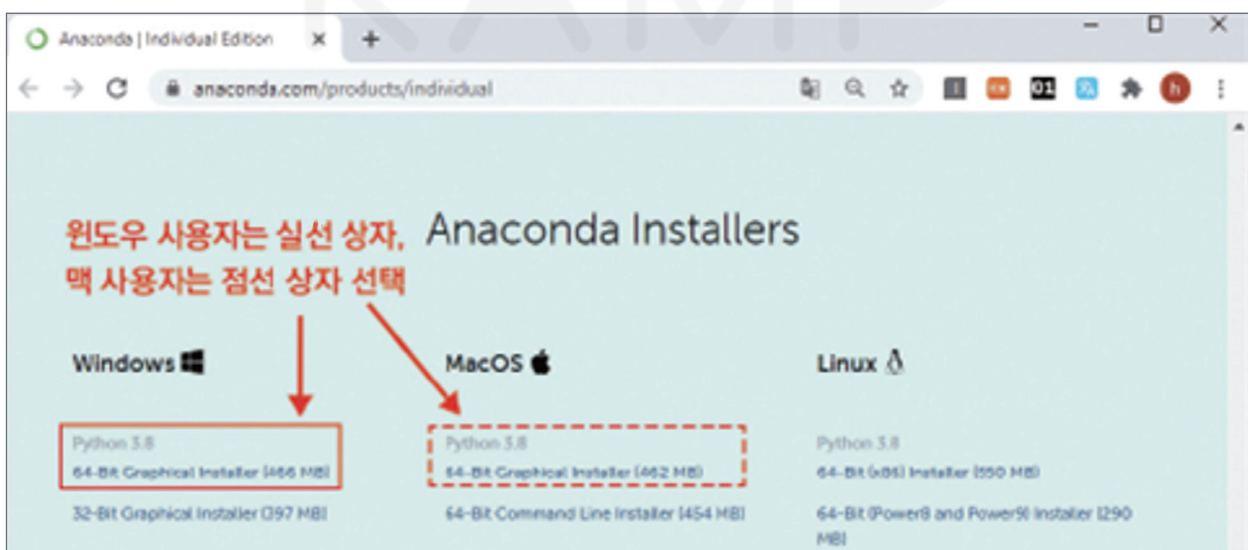
아나콘다란 파이썬 등의 데이터 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 효율적으로 사용할 수 있으며, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고, 데이터를 분석할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



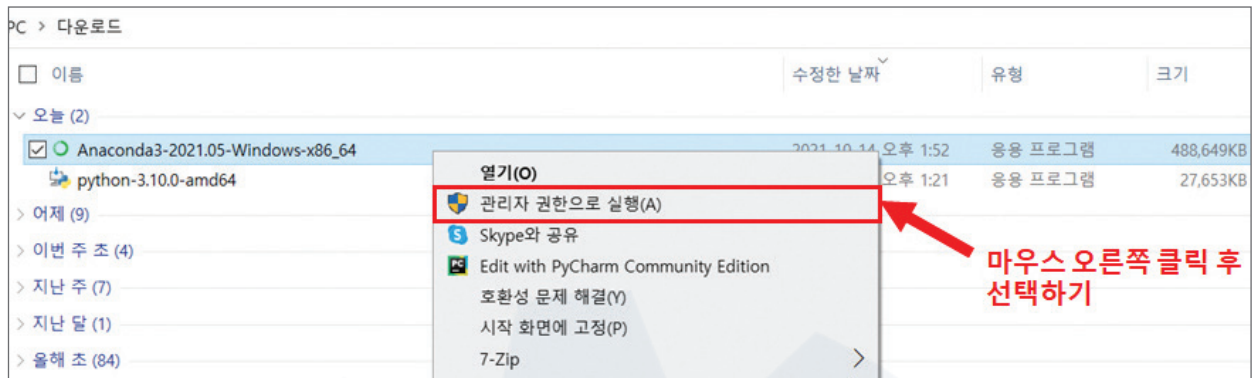
② 스크롤을 내려 다음의 화면을 확인한 후, 로컬 컴퓨터 사용 환경에 맞는 아나콘다 설치 파일을 다운로드한다.

- ▶ Windows : 64-Bit Graphical Installer
- ▶ MacOS : 64-Bit Graphical Installer

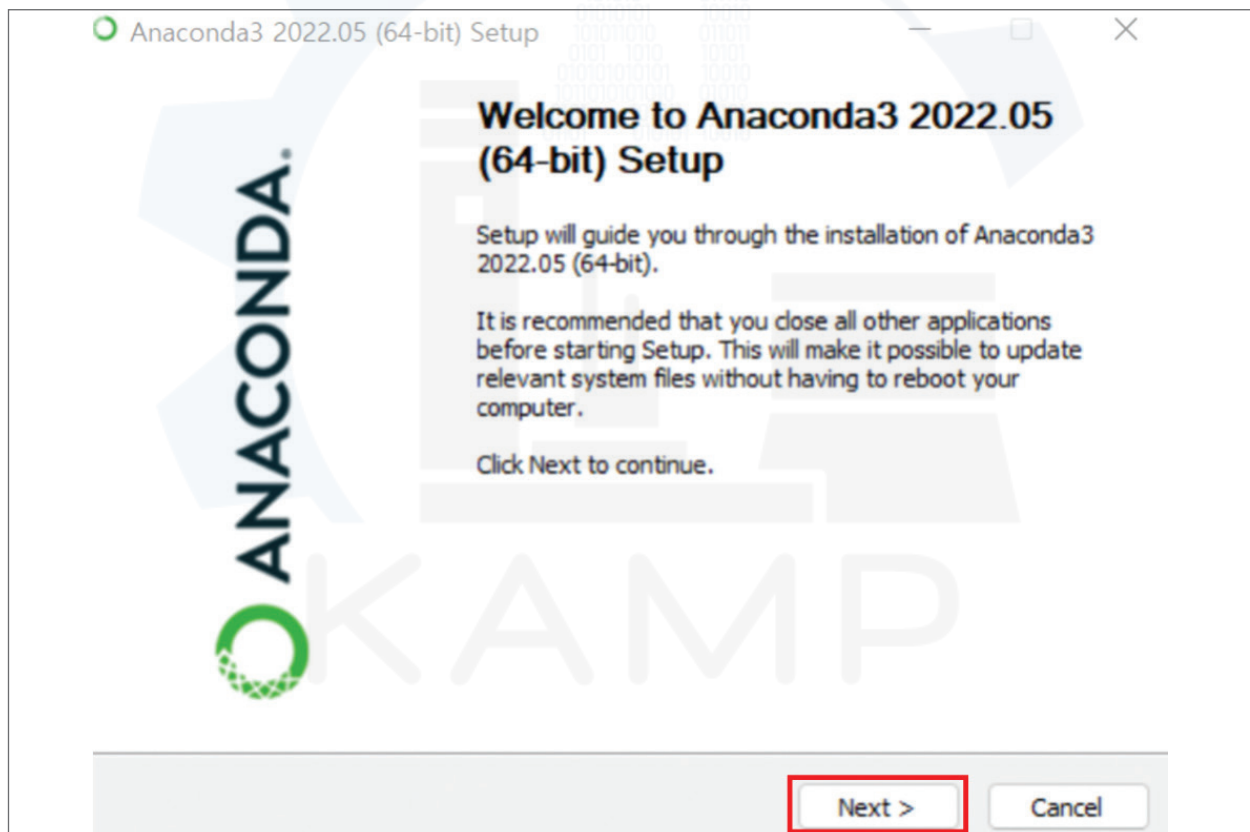


- ③ 다운로드한 아나콘다 설치 파일에서 마우스 오른쪽을 클릭한 후, 방패모양의 ‘관리자 권한으로 실행’을 클릭한다.

▶ (예) ‘다운로드’ 폴더로 아나콘다를 다운로드한 경우

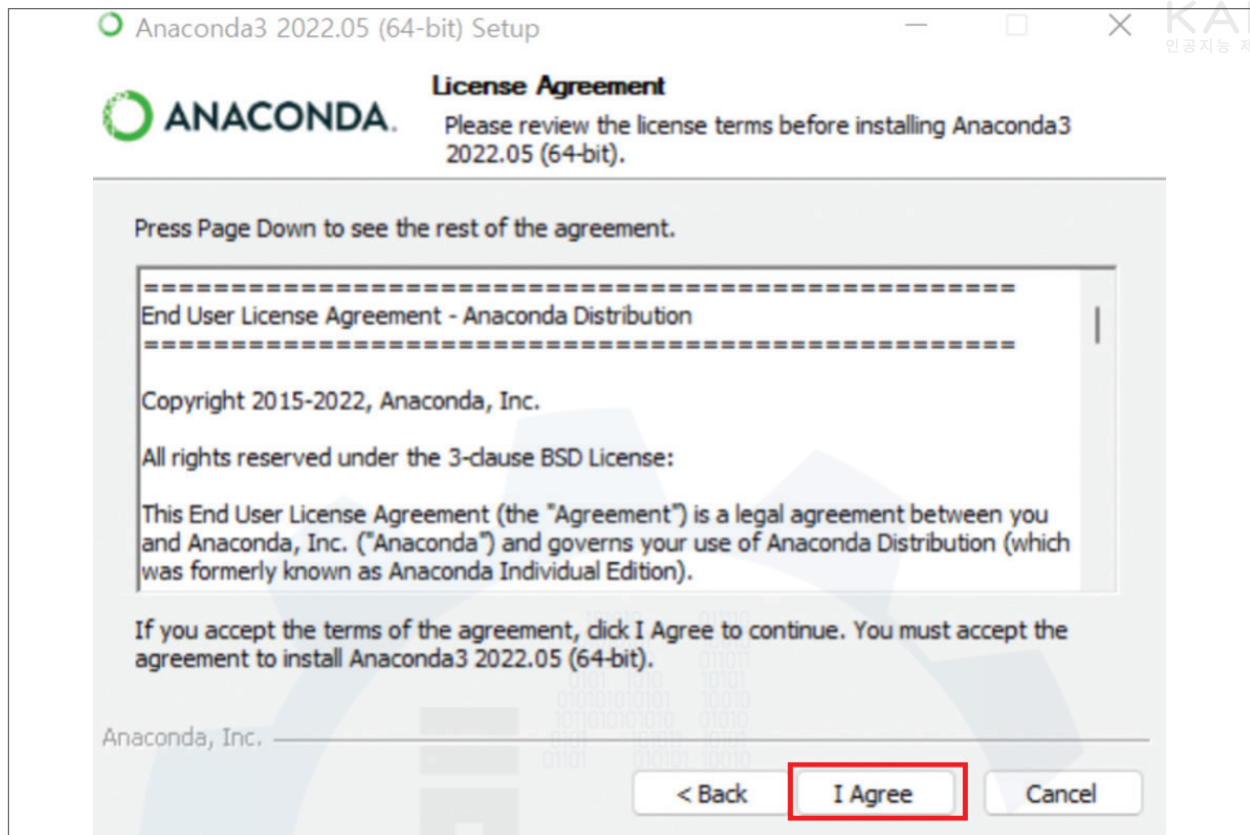


- ④ 설치 파일을 실행한 다음, ‘Next’ 버튼을 클릭한다.

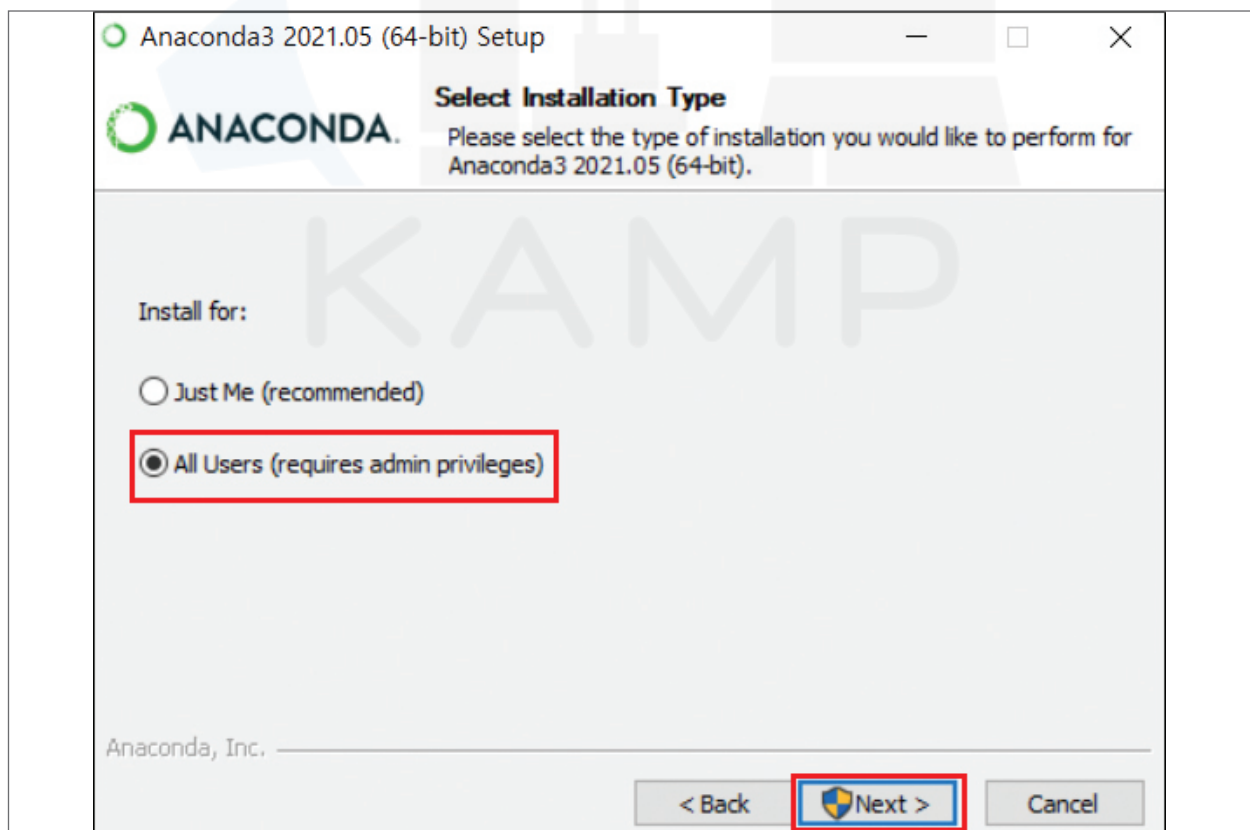




⑤ 다음 단계에서 아래와 같은 창이 나타나면, 'I Agree'를 선택한다.

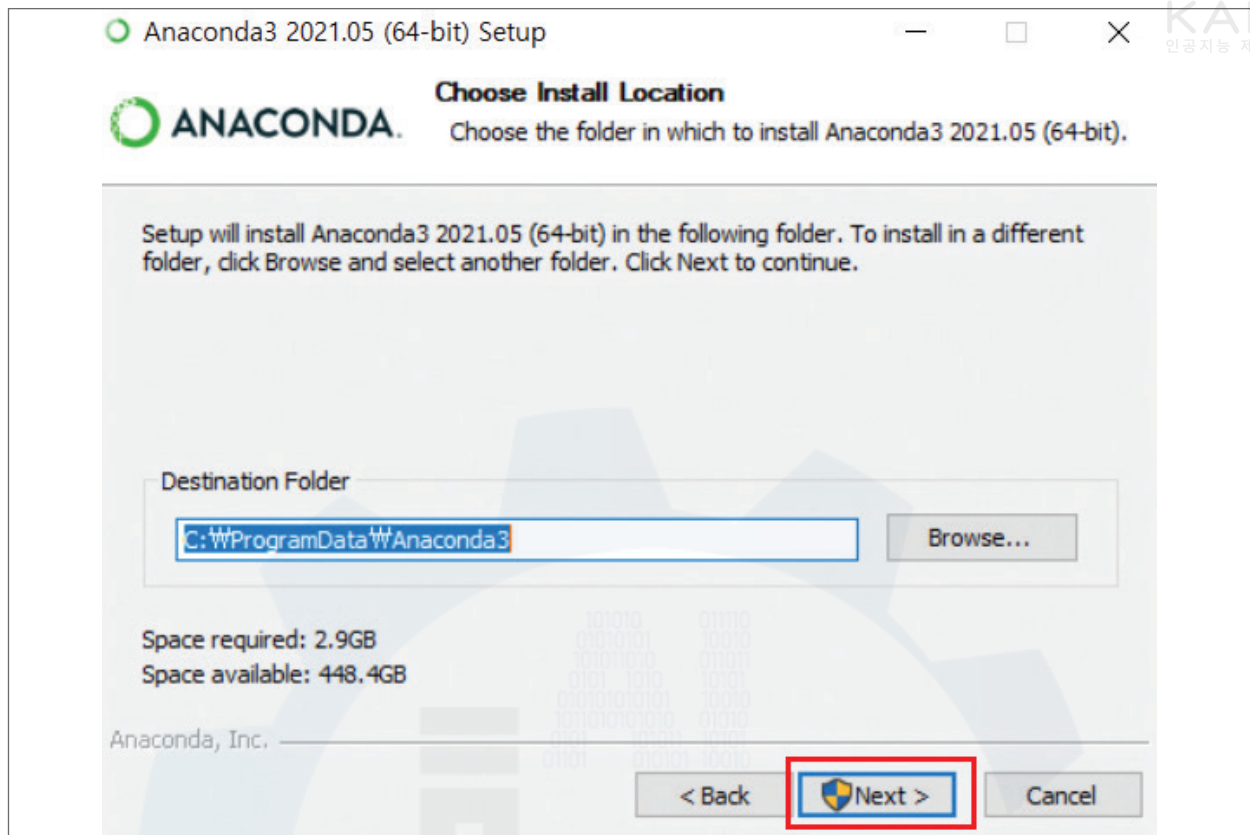


⑥ 아래와 같은 창이 나타나면, 'All Users'를 선택하여 다음 단계로 진행한다.

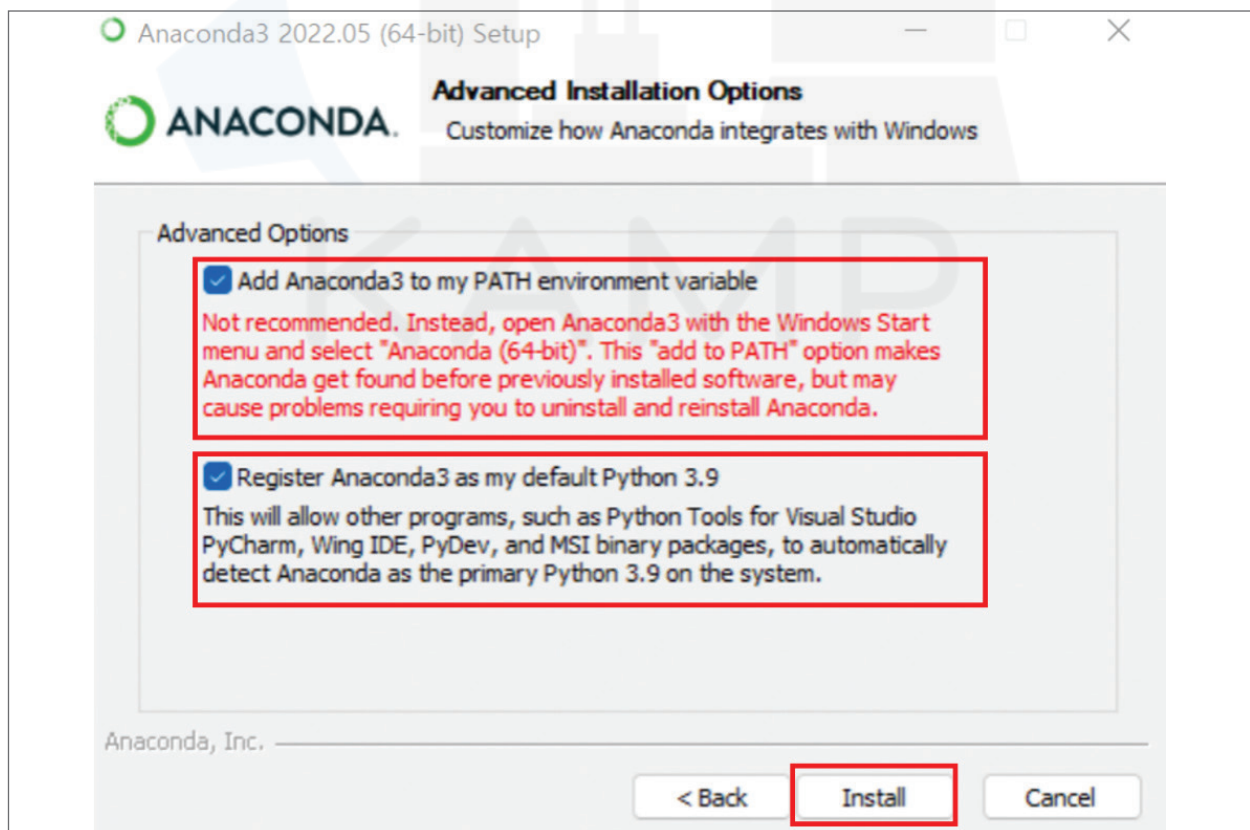




⑦ 다운로드할 경로를 설정한 뒤, 아래의 'Next'를 클릭한다.

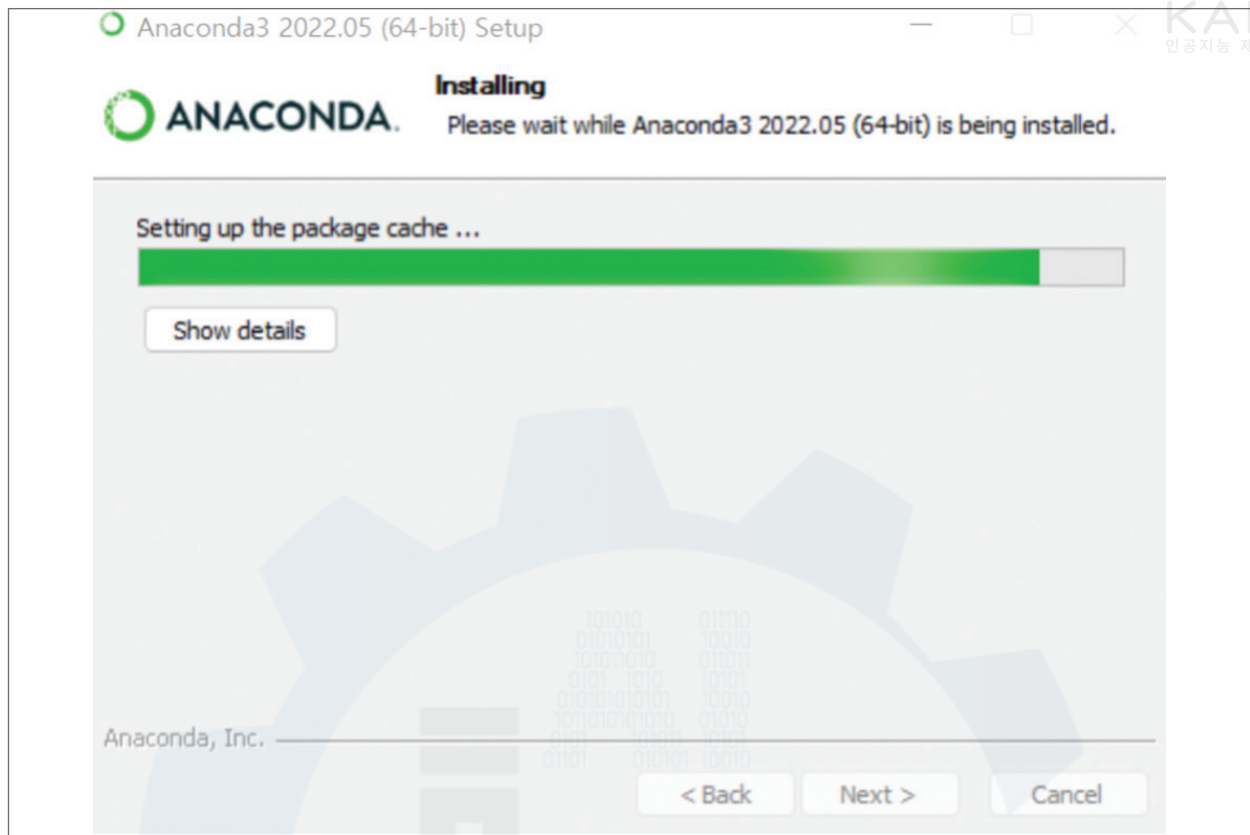


⑧ 고급 옵션 선택창에서 아래와 같이 모두 선택 후, 'Install'을 클릭한다.

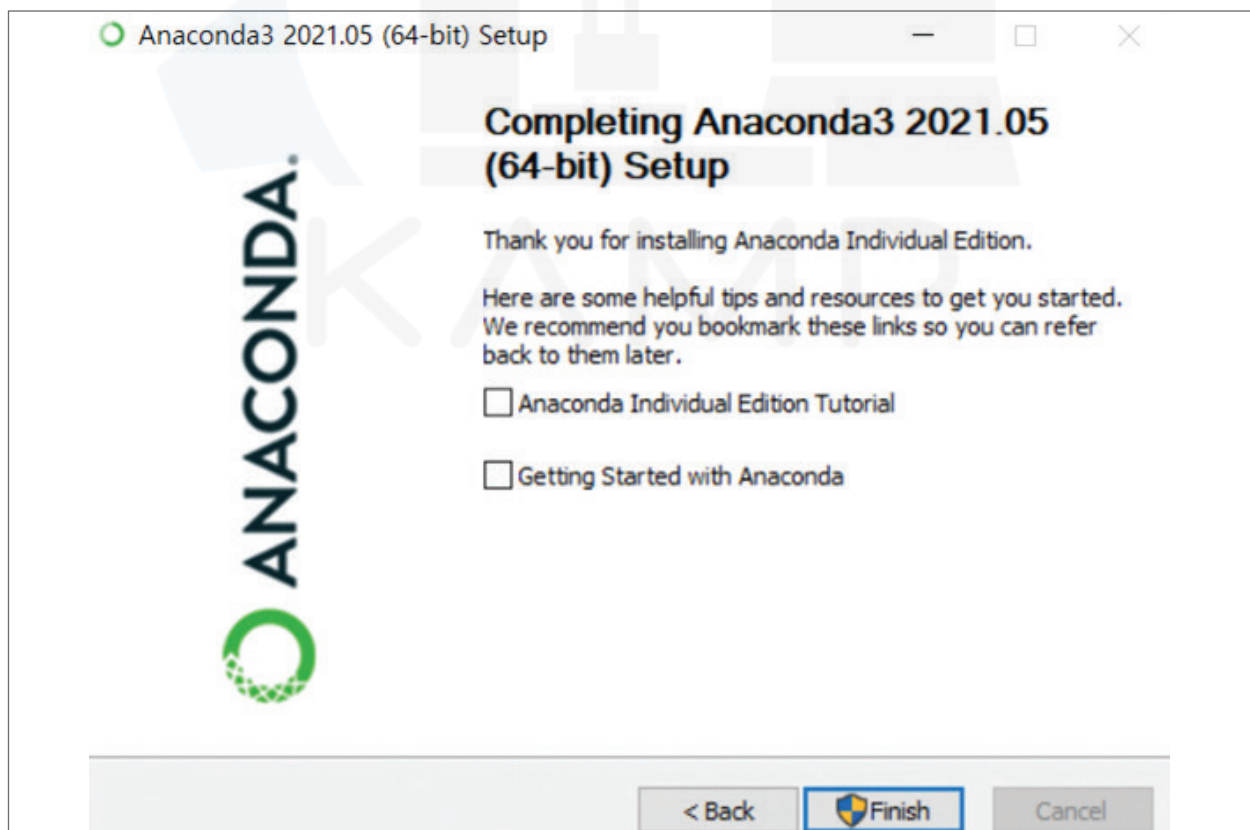




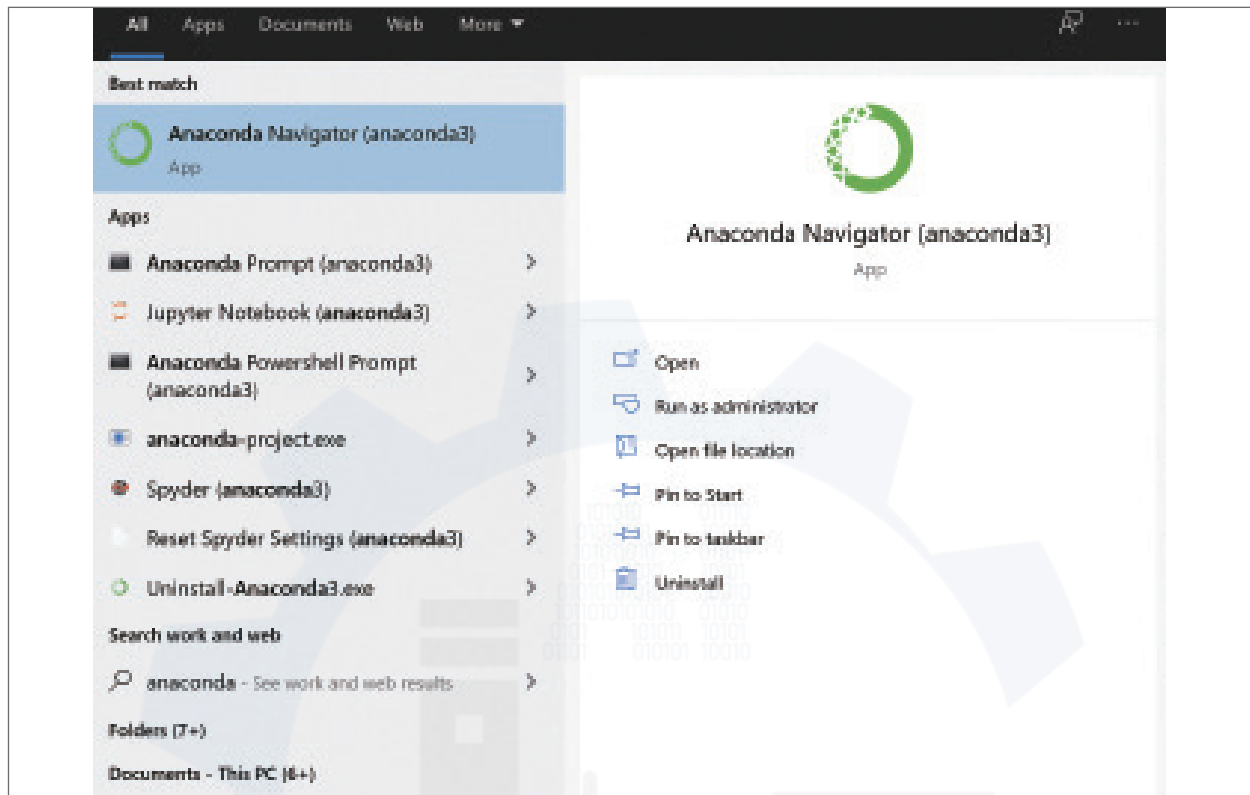
⑨ 다음과 같은 설치창으로 넘어가면, 완료가 될 때까지 대기한다. (5분이상 소요)



⑩ 마지막 화면에서 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



- ⑪ PC의 '시작 버튼(Windows logo)'을 눌러서 화면과 같이 anaconda prompt가 설치되었는지 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.

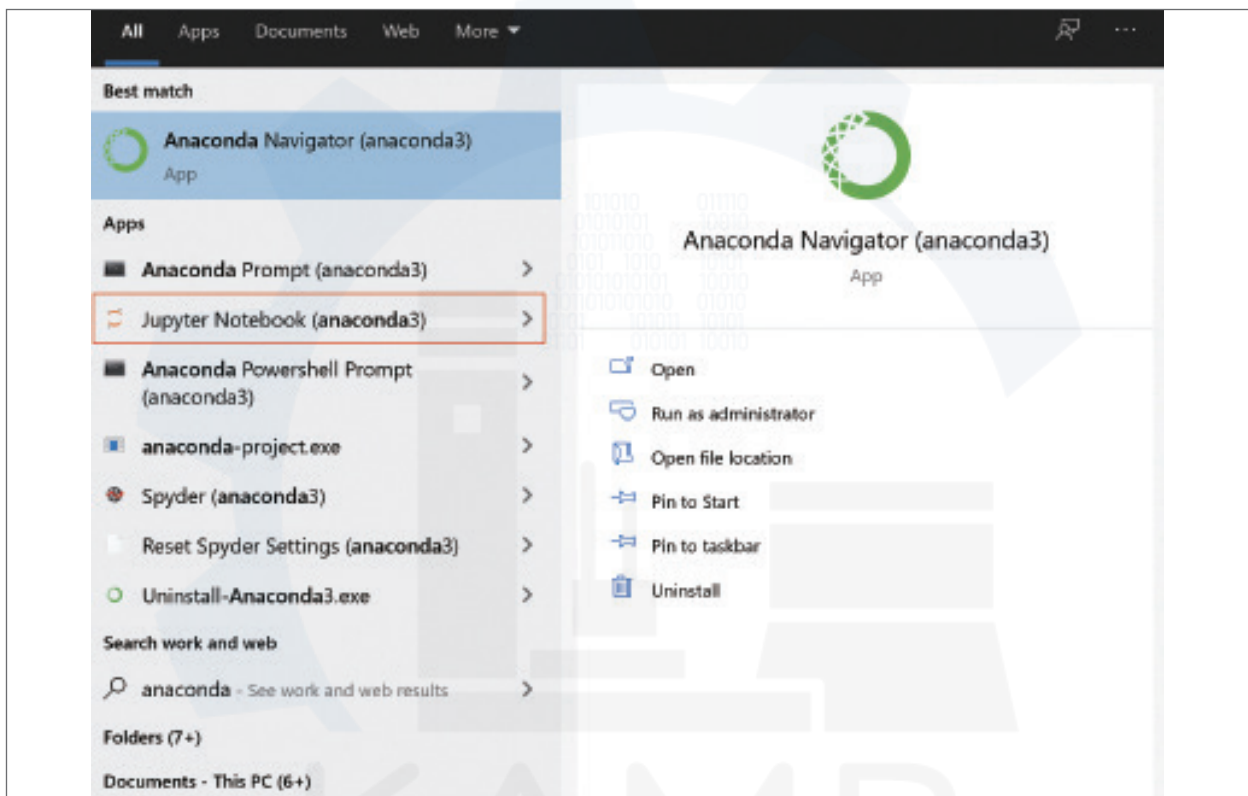




3. 주피터 노트북 (Jupyter Notebook) 실행

주피터 노트북은 사용자가 코딩(분석 코드 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 주피터 노트북을 활용하는 방법을 안내한다.

- ① PC의 '시작 버튼(Windows logo)'을 눌러서 화면과 같이 'anaconda'를 검색한 후 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.





② Jupyter Notebook을 실행하면, 아래와 같이 2개의 윈도우가 실행된다.

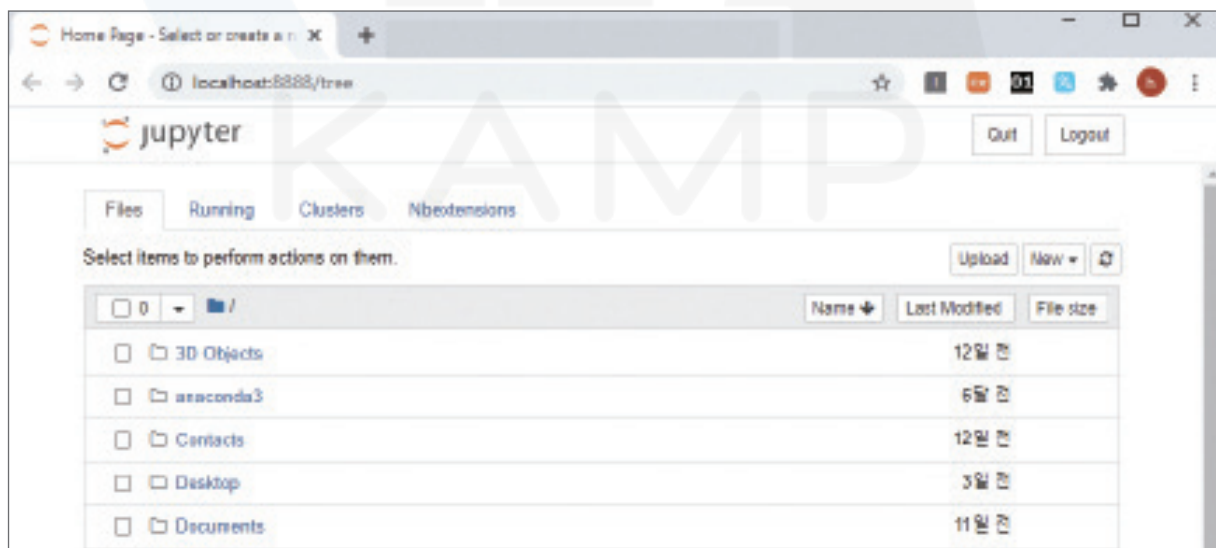
(1) 아래 검은색 배경의 화면은 주피터 노트북이 실행되는 환경 상태를 나타내는 상태 표시 창이다. 주피터 노트북을 사용하는 동안에는 종료하면 안된다.

```

Jupyter Notebook (anaconda3)
[I 12:39:38.566 NotebookApp] JupyterLab extension loaded from C:\Users\leedaeyoung\Anaconda3\lib\site-packages\jupyterlab
[I 12:39:38.566 NotebookApp] JupyterLab application directory is C:\Users\leedaeyoung\Anaconda3\share\jupyter\lab
[I 12:39:38.627 NotebookApp] Serving notebooks from local directory: C:\Users\leedaeyoung
[I 12:39:38.628 NotebookApp] The Jupyter Notebook is running at:
[I 12:39:38.628 NotebookApp] http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 12:39:39.053 NotebookApp]

To access the notebook, open this file in a browser:
file:///C:/Users/leedaeyoung/AppData/Roaming/jupyter/runtime/nbserver-6568-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
  
```

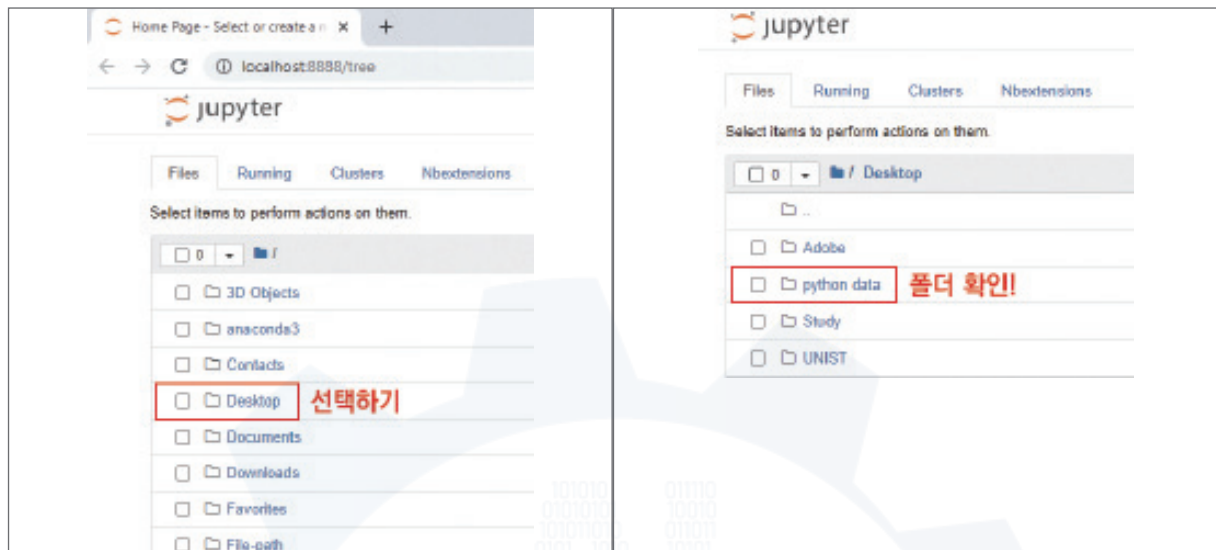
(2) 기본 브라우저(크롬, 엣지 등)에서 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야 한다.)



③ 데이터 분석을 진행할 폴더를 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성한 다음, 파이썬 코드 실행하고 저장한다.

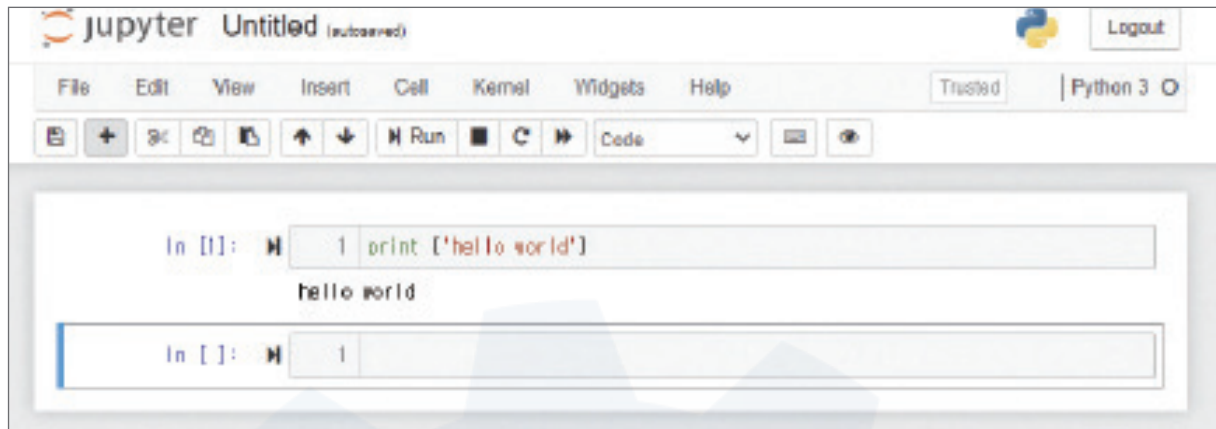
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



(2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world')를 입력한 다음, 상단의  Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '*.ipynb'로 저장된다.





4 부록_데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한 6가지 특성에 대하여 논의하였다. (완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

▶ 데이터 품질 분석을 위한 라이브러리 불러오기

```
import pandas as pd
import datetime
```

▶ 데이터 품질 분석을 위한 데이터 불러오기

```
df = pd.read_csv('Resource_Management_Process.csv')
```

▶ 완전성 품질지수 = (1 - (결측치 데이터 수/ 전체 데이터 수)) × 100

데이터의 완전성(Completeness) 품질지수를 계산하기 위해서는 결측치와 전체 데이터 수를 구해야 한다.

- ① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 Null 값의 비율을 확인하여 삭제한다. 이 데이터는 30%를 넘는 열이 존재하지 않으므로 열을 제거하지 않는다.

```
perc = 30

print('[step 1-1]')
print(round(df.isnull().sum()/len(df)*100, 2))
print('[step 1-2]')
print(df.isnull().sum()/len(df)*100>perc)
```

```
[step 1-1]
Datetime      0.0
Production    0.0
Temperature    0.0
Humidity       0.0
Power_Cost     0.0
DoW           0.0
Worker_power  0.0
Man_Cost       0.0
Power_Usage    0.0
dtype: float64
[step 1-2]
Datetime      False
Production    False
Temperature    False
Humidity       False
Power_Cost     False
DoW           False
Worker_power  False
Man_Cost       False
Power_Usage    False
dtype: bool
```



- ② DataFrame의 isnull() 함수와 sum() 함수를 이용해 전체 데이터에서 결측치 데이터 수를 확인할 수 있다.

```
print("[step 2-1]")
print(df.isnull().head())
print("[step 2-2]")
print(df.isnull().sum())
print("[step 2-3]")
cmpt_len = df.isnull().sum().sum()
print(cmpt_len)
```

```
[step 2-1]
Datetime Production Temperature Humidity Power_Cost DoW \
0 False False False False False False
1 False False False False False False
2 False False False False False False
3 False False False False False False
4 False False False False False False
Worker_power Man_Cost Power_Usage
0 False False False
1 False False False
2 False False False
3 False False False
4 False False False
[step 2-2]
Datetime 0
Production 0
Temperature 0
Humidity 0
Power_Cost 0
DoW 0
Worker_power 0
Man_Cost 0
Power_Usage 0
dtype: int64
[step 2-3]
0
```

- ③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

```
completeness = (1-cmpt_len/len(df))*100
print("결측치 = %d개 \n완전성 품질지수 : %.2f%%"%(cmpt_len,completeness))
```

```
결측치 = 0개
완전성 품질지수 : 100.00%
```

▶ 유일성 품질지수 = (유일한 데이터 수 / 전체 데이터 수) × 100

데이터의 유일성(Uniqueness) 품질 지수를 계산하기 위해서는 유일한 데이터 수와 전체 데이터 수를 구해야 한다.

```
unique_len = len(df["Datetime"].unique())
uniqueness = (unique_len/len(df))*100
print("유일성 지수 : %.2f%%"%(uniqueness))
```

```
유일성 지수 : 97.65%
```



▶ 유효성 품질지수 = (유효성 만족 데이터 수 / 전체 데이터 수) × 100

데이터의 유효성(Validity) 품질지수를 계산하기 위해서는 유효성 만족 데이터 수와 전체 데이터 수를 구해야 한다.

- ① 데이터의 측정 시각은 날짜 시간의 표현이 “연-월-일 시:분”으로 정해진 datetime 형식인지 확인하는 함수를 구현한다.

```
def validate_date(date_text):
    try:
        datetime.datetime.strptime(date_text, "%Y-%m-%d %H:%M")
        return True
    except ValueError:
        print("Incorrect data format({0}), should be YYYY-MM-DD".format(date_text))
        return False
```

- ② 유효성 품질지수 값은 다음과 같다.

```
valid_score_list = []

for i in range(len(df)):
    validate_score = validate_date(df["Datetime"][i])
    valid_score_list.append(validate_score)

validity = sum(valid_score_list)/len(df)*100

print(f'유효성 품질지수 : {round(validity, 2)}%')
```

유효성 품질지수 : 100.0%

▶ 일관성 품질지수 = (일관성 만족 데이터 수 / 전체 데이터 수) × 100

데이터의 일관성(Consistency) 품질지수를 계산하기 위해서는 일관성 만족 데이터 수와 전체 데이터 수를 구해야 한다.



① 데이터의 각 column 별 데이터형이 일관성 있는지 검사하고, 일관성 품질지수를 측정한다.

```
consistency_score_list = []

for i in range(len(df)):
    consistency_score_1 = type(df["Datetime"][i]) == str
    consistency_score_2 = df["Production"][i].dtype == df.dtypes[1]
    consistency_score_3 = df["Temperature"][i].dtype == df.dtypes[2]
    consistency_score_4 = df["Humidity"][i].dtype == df.dtypes[3]
    consistency_score_5 = df["Power_Cost"][i].dtype == df.dtypes[4]
    consistency_score_6 = type(df["DoW"][i]) == str
    consistency_score_7 = df["Worker_Power"][i].dtype == df.dtypes[6]
    consistency_score_8 = df["Man_Cost"][i].dtype == df.dtypes[7]
    consistency_score_9 = df["Power_Usage"][i].dtype == df.dtypes[8]
    consistency_score_list.extend([consistency_score_1, consistency_score_2, consistency_score_3, consistency_score_4, consistency_score_5, consistency_score_6, consistency_score_7, consistency_score_8, consistency_score_9])

consistency = (sum(consistency_score_list)/len(df)/9)*100

print(f'일관성 품질지수 : {round(consistency, 2)}%')
```

일관성 품질지수 : 100.0%

▶ 정확성 품질지수 = (1 - (정확성 위배 데이터 수 / 전체 데이터 수)) × 100

- 데이터의 정확성(Accuracy) 품질지수를 계산하기 위해서는 정확성 위배 데이터 수와 전체 데이터 수를 구해야 한다.
- 본 실습의 데이터셋은 2021년 2월 7일~10월 22일에 수집되었다. 따라서, 각 데이터셋의 측정 시각이 해당 기간에 포함되는지 여부를 통해 정확성을 검사한다.

- ① 'datetime.date()' 함수는 연도, 달, 일을 input으로 받고 datetime을 만든다.
pd.Timestamp 함수는 pandas의 timestamp를 생성하는 함수로, pandas의 datetime을 비교하는데 비교하기 쉽게 도와준다.

```
accuracy_score_list = []
d0 = pd.Timestamp(datetime.date(2021,2,7)) #date 객체1
d1 = pd.Timestamp(datetime.date(2021,10,22)) #date 객체2
```

② 앞서 정의한 d0와 d1을 이용해 해당 기간 포함 여부를 확인한다.

```
con1 = pd.to_datetime(df["Datetime"]) >= d0
con2 = pd.to_datetime(df["Datetime"]) <= d1
vald_df = df[con1&con2]
accuracy_score = len(df) - len(vald_df)
accuracy_score_list.append(accuracy_score)
print(f'{df} 수집된 날짜를 벗어나는 데이터 수: {accuracy_score}')
```

수집된 날짜를 벗어나는 데이터 수: 0



③ 정확성 품질지수 값은 다음과 같다.

```
accuracy = 100 - sum(accuracy_score_list) / len(accuracy_score_list)
print(f'정확성 품질지수 : {round(accuracy, 2)}%')
```

정확성 품질지수 : 100.0%

▶ 무결성 품질지수 = (유일성, 유효성, 일관성 지수 중 100% 지수 개수 / 3) × 100

이전 단계에서 계산된 유일성, 유효성, 일관성 품질지수를 확인한다.

```
print('유일성 품질지수: %d%%' % uniqueness)
print('유효성 품질지수: %d%%' % validity)
print('일관성 품질지수: %d%%' % consistency)
```

유일성 품질지수: 97%
유효성 품질지수: 100%
일관성 품질지수: 100%

유일성, 유효성, 일관성 지수 중 100%인 품질지수 수를 구하고, 무결성 품질지수를 계산한다.

```
integrity = 2/3 * 100
print(f'무결성 품질지수: %d%%' % integrity)
```

무결성 품질지수: 66%

▶ 가중치 지수 = 품질지수 × 가중치

안전성, 유일성, 유효성, 정확성, 무결성 품질지수에 각 0.2, 0.2, 0.2, 0.15, 0.15, 0.1의 가중치를 부여해 각 품질지수를 계산한다.

```
print(f'안전성 가중치 품질지수 : {round(completeness*0.2, 2)}%')
print(f'유일성 가중치 품질지수 : {round(uniqueness*0.2, 2)}%')
print(f'유효성 가중치 품질지수 : {round(validity*0.2, 2)}%')
print(f'일관성 가중치 품질지수 : {round(consistency*0.15, 2)}%')
print(f'정확성 가중치 품질지수 : {round(accuracy*0.15, 2)}%')
print(f'무결성 가중치 품질지수 : {round(integrity*0.1, 2)}%')
```

안전성 가중치 품질지수 : 20.0%
유일성 가중치 품질지수 : 19.53%
유효성 가중치 품질지수 : 20.0%
일관성 가중치 품질지수 : 15.0%
정확성 가중치 품질지수 : 15.0%
무결성 가중치 품질지수 : 6.67%



▶ 데이터 품질지수 = ∑가중치 지수

각 품질지수에 가중치를 적용해 종합적인 데이터 품질지수를 계산한다.

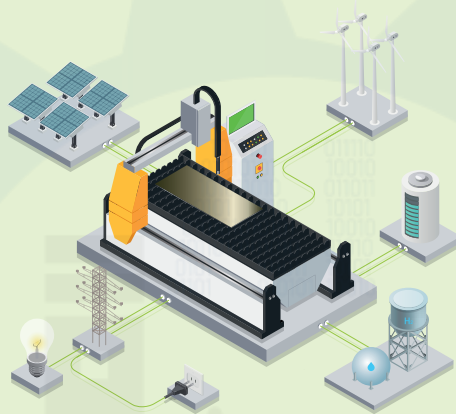
```
data_quality = completeness *0.2 + uniqueness *0.2 + validity *0.2 + consistency *0.15 + accuracy *0.15 + integrity *0.1
```

```
print(f'데이터 품질지수 : {round(data_quality, 2)}%')
```

데이터 품질지수: 96.2%

구분	품질지수	가중치	가중치 지수	오류율
완정성 (누락)	100%	20%	20%	0%
유일성 (중복)	97.65%	20%	19.53%	0.47%
유효성 (유효)	100%	20%	20%	0%
일관성 (표현)	100%	15%	15%	0%
정확성 (실제)	100%	15%	15%	0%
무결성	66.67%	10%	6.67%	3.33%
품질지수	-	100%	96.2%	3.8%

[표 1] 데이터 유효성 검증 결과



「표면처리 자원최적화 AI 데이터셋」
분석실습 가이드북